PROTOCOL WHITEPAPER

Dynamo

The Interplanetary AI Economic and Financial Layer

One budget. Many agents.
Payment streamed by the second.
Across the universe.

Dynamo Protocol Team

September 2026

dynamo.zone

Contents

Abstract.....3
1. Introduction.....3
2. Background and Related Work.....5
3. Design Doctrines.....7
4. System Model and Architecture.....8
5. The Core Protocol, Formally.....10
6. Security Properties and Analysis.....19
7. Implementation and Evaluation.....22
8. The Interplanetary AI Economic and Financial Layer.....26
9. The DYNAMO Utility Token.....30
10. Discussion.....33
11. Conclusion.....34
References.....35
Appendix A - Notation.....37
Appendix B - Demonstrated Behaviors.....38
Appendix C - Implementation Scope and Reproducibility.....38
Appendix D - Economic analysis and evaluation criteria.....39

Abstract

Dynamo is the Interplanetary AI Economic and Financial Layer, a proposed architecture for economic coordination among AI agents and service providers across heterogeneous funding systems and, prospectively, disconnected execution domains. Its current protocol foundation represents delegated authority as a tree of capped streams, records metered work in cumulative signed vouchers, and settles accepted increments through funding-specific backends. We define the transition model and prove conditional results for accounting-cap conservation, replay safety, fee rounding, and fixed-quote asset conversion. An admission lemma identifies the additional reservation and integration conditions needed to bound obligations before execution. We analyze rate envelopes, checkpoint cadence, revocation, closure, refund funding, and cross-system crash recovery, distinguishing safety from payment liveness. Authenticated service observations support configured delivery policies but do not establish measurement truth or general quality verification. Public contract semantics are separated from project-reported proprietary runtime behavior and testnet demonstrations; reported test counts are not treated as independent replication. A comparative evaluation protocol specifies adversarial workloads, reproducibility artifacts, cost accounting, and timing measurements. Orbital training and inference demonstrations and data-center research motivate an extension based on disjoint local authority and delayed reconciliation, whose requirements remain distinct from terrestrial contract safety. DYNAMO is proposed as a utility token for protocol governance, accountable participation, and execution fees if a dedicated ledger is justified; its functions require separate implementation and conformance analysis. The contribution is the formalized composition and analysis of these primitives, rather than novelty of the underlying cryptography or a proof of commercial adoption.

Keywords: AI agent economy; distributed systems; delegated authority; accounting invariants; cumulative vouchers; streaming settlement; utility token; orbital computing.

1. Introduction

Dynamo's vision is an economic and financial layer through which AI agents and services can produce, purchase, and exchange useful work. Payment and budget infrastructure are foundational capabilities within that vision. The organizing message is **one budget, many agents, payment streamed by the second, across the universe**. The engineering task is to make each part of that message precise: who authorizes work, what becomes payable, how several providers share funding, when funds become usable, and which guarantees survive failures or disconnection.

An AI developer may supply an inference endpoint, a Model Context Protocol (MCP) tool, a data feed, a game-generation pipeline, or a video service. Each can consume paid inputs before its own customer payment becomes available. Dynamo connects measured delivery to payment claims and gives multiple service providers a common framework for delegated authority and reconciliation. The intended result is shorter cash-conversion delays, more economical small payments, and less custom payment coordination code. Whether those benefits materialize is measured at the deployment and workload level, rather than inferred from a protocol fee alone.

Terminology. An AI agent is software that selects and performs actions under delegated authority. A service provider supplies a tool, model, data feed, or other measurable work. A budget is an authorization ceiling; a stream is a service-specific allocation within it. Metering records billable quantities. A voucher is a signed cumulative payment claim, and settlement is the funding backend's acceptance and execution of its payable increment. Signing a voucher and receiving usable funds are different events. Escrow holds assets subject to contract rules. An attestation is a signed observation whose truth depends on its source. These definitions apply throughout the paper.

The **Dynamo protocol** specifies coordination and settlement behavior. **DYNAMO** denotes the proposed ecosystem token described in Section 9. The protocol can operate without that token. Its micro-USD accounting

unit, a service's settlement asset, and DYNAMO are distinct: the first measures an amount, the second transfers payment, and the third has proposed participation and infrastructure functions.

1.1 The problem and the opportunity

Software increasingly participates in economic workflows by invoking paid APIs and purchasing intermediate outputs. Research on the agentic economy identifies programmatic consumer-service interactions as a potential source of changes in market structure [10]. METR's 2025 study reports an approximately seven-month doubling time in the human-duration tasks that tested agents could complete with 50% success; that is a capability measure, not adoption or financial reliability [14]. Multi-agent research identifies design, coordination, and verification failures [21]. Gartner's projected project-cancellation rate is a forecast rather than an observed failure rate [15]. The motivation is therefore a combination of expanding technical capability and unresolved operational needs.

Existing payment and authorization systems already solve important parts of this problem. The HTTP payment protocol x402 includes capped usage-based payments and batch settlement; the Machine Payments Protocol (MPP) supports machine payments and sessions; the Agent Payments Protocol (AP2) addresses mandates and transaction evidence [18, 23-27]. Dynamo's design question is how to compose metering, recursive spending authority, and financial records around a shared workflow. The paper does not rely on competing systems remaining limited to fixed, bilateral payments.

1.2 Provider requirements

Providers need payment for delivered work, predictable processing overhead, and a way to compose paid inputs within application logic. Dynamo addresses these requirements with checkpointed claims, a disclosed fee mechanism, and scoped delegation. The evaluation targets are time from eligible delivery to usable funds, total lifecycle payment cost, and integration and reconciliation effort. Per-second metering does not imply per-second ledger finality.

At the documented 1% escrow fee, approximately 99% of gross identity-priced settlement remains before gas and other costs; this is neither a profit-margin claim nor an immutable fee. The comparison must include aggregated card billing and other low-cost payment paths. Cards also have programmable APIs. Dynamo's technical distinction is its proposed composition of token-native settlement, nested authority, and evidence-linked accounting across independent services. Supported funding methods and jurisdictions constrain cross-border reach.

These requirements explain the practical motivation for the protocol. Appendix D provides the economic definitions and worked examples; the formal properties in Section 5 do not depend on a favorable market outcome.

1.3 User and operator requirements

AI users need a way to commission a result without giving every agent unrestricted purchasing authority. A parent budget can allocate separate capped streams to models, APIs, data providers, or subcontracted agents. Signed usage records provide a basis for explaining who consumed what. Configured delivery conditions can determine billable units, and revocation can limit further authority under the applicable enforcement rules.

Dynamo makes *gross settlement above the accounting cap* impossible under the contract model in Section 5. Preventing *all external expenditure* additionally requires non-bypassable admission, bounded cost reservations, and coordinated access to shared headroom. Provider credentials outside that boundary can incur unrelated liabilities. A signature proves the source of a quality observation; it does not independently prove the quality of a game, video, or model response. These boundaries let users choose controls appropriate to the actual workload.

1.4 Scope and contribution

The present protocol turns a parent ceiling into a tree of capped, revocable streams. Metering generates cumulative signed claims; the funding backend settles the accepted increments. Control Mode records this accounting without moving money. Escrow connects it to contract-held assets. The card backend connects the common object model to processor authorizations and captures, with different finality and reversal rules.

Service adapters can price compute time, calls, bytes, rendered frames, energy, or human work. They convert those measurements into a canonical micro-USD accounting denomination; physical units are not numerically interchangeable. Section 5 defines the financial kernel and its assumptions. The longer-term economic scope includes service discovery, reputation, risk coordination, and remote resource exchange, whose maturity differs from that of the kernel.

The paper makes six contributions:

1. A compositional budget model with node, subtree, and aggregate accounting conservation under stated transitions (Sections 4 and 5.1-5.3).
2. A cumulative-voucher model with a telescoping no-duplicate-settlement result and explicit signer and domain requirements (Section 5.5).
3. A delivery-policy model separating authenticated observations, envelope clipping, implemented refunds, and proposed funded holdbacks (Section 5.8).
4. A shared funding interface whose control, escrow, and card semantics remain explicit (Section 4.2).
5. A reproducible technical and commercial evaluation framework covering implementation scope, failure cases, costs, and provider cash flow (Sections 7 and 8).
6. A grounded path toward interplanetary economic coordination and the stated DYNAMO utility design, with separate readiness conditions for each extension (Sections 8 and 9).

Reading guide. Section 1 motivates the system requirements. Sections 2-6 cover related work, the model, formal properties, and security analysis. Section 7 describes implementation evidence and the evaluation methodology. Section 8 examines architectural extensions, and Section 9 specifies the token design. Appendices provide notation, replication requirements, and the supporting economic analysis.

2. Background and Related Work

Off-chain payment channels. Bitcoin and Ethereum provide the underlying ledger models [1, 2]. Lightning uses signed off-chain updates with later on-chain enforcement, and payment-channel-hub work such as BlindHub studies conditional multi-party payments and privacy [3, 22]. Dynamo adopts the established idea of settling cumulative signed state. Its emphasis is a shared budget represented as an allowance tree, rather than a claim to have invented multi-party payments. Unlike a mature payment-channel protocol with a dispute period, the vault permits a budget owner to revoke and close before an outstanding voucher is submitted; this payer-safety/payee-liveness trade-off matters in any comparison.

Streaming money. Sablier provides token-streaming primitives [20]. Dynamo combines a cumulative rate envelope with metered claims: wall-clock time bounds what may settle, but time alone does not create the next payable voucher. A service adapter determines what constitutes billable work. The distinction is between an authorization envelope and a delivery-dependent accounting process, not between protocols that do and do not support time.

Agent payment standards. The 2025 x402 whitepaper defines the initial HTTP payment design [9]. The September 2026 documentation describes additional schemes and extensions. Its `upto` scheme authorizes a maximum and settles an amount determined by actual usage; batch settlement and signed offers/receipts expand the evidence and efficiency surfaces [23-26]. MPP, introduced by Stripe and Tempo in March 2026, coordinates machine payments across methods and supports microtransactions and recurring use; the public MPP site also describes an evolved sessions implementation [18]. AP2 v0.2 binds checkout and payment mandates to receipts and supports human-present and autonomous modes. It discusses agent-to-agent delegation conceptually while leaving it outside the specification [27]. Dynamo's proposed contribution is a particular composition of recursive allocation, accounting caps, and evidence-linked settlement. Interoperability with a standard must be demonstrated against a named version and scheme; an HTTP 402 response alone is not sufficient.

Identity and validation. ERC-8004 specifies identity, reputation, and validation registries [13]. Xiong et al.'s empirical study covers Ethereum, BSC, and Base through 13 May 2026 and reports substantial weaknesses in the sampled identity and reputation records [12]. This is evidence about a dated sample, not a timeless verdict on all registry deployments. Vaziry et al. integrate ledger-anchored agent identities with A2A and x402 payments [11]. Dynamo may complement these approaches by attaching settlement records to interactions, but paid interactions can themselves be manufactured or collusively recycled; payment evidence is not automatically honest reputation.

Chargebacks and the evidence gap. Agent-initiated commerce raises distinct questions of authority, delivery, and liability. AP2 includes dispute evidence, and Visa's Trusted Agent Protocol specifies signed agent requests, intent, and related recognition and payment information [27, 28]. Mastercard's Verifiable Intent is a complementary authorization initiative [29]. Dynamo's opportunity is to record metered service performance and connect it to financial state. Whether a particular evidence bundle satisfies a dispute process depends on that process, jurisdiction, contractual terms, and externally anchored identities. Cryptographic integrity alone neither establishes legal admissibility nor allocates liability.

The counter-commoditization problem. Basic budget counters and quotas can be implemented in gateways and observability tools. Their enforceability depends on architecture, not category labels. Dynamo seeks differentiation through the coupling of those controls to settlement and a shared multi-service budget. This is a product and systems hypothesis requiring comparison against well-engineered alternatives, including transactional gateway counters and reserved-spend implementations; it is not a theorem that competitors cannot add settlement.

Agent tooling. MCP standardizes tool integration [17]. Dynamo's `monetize()` surface and gateway adapters address service monetization and spend control. They allow tool-specific measurement and pricing to be connected to a common payment interface. The extent of unmet monetization demand is a market question, not an implication of protocol compatibility.

Fee markets and transaction ordering. EIP-1559 analysis and research on miner/maximal extractable value provide relevant economic context [7, 8]. Signed vouchers constrain a claim's contents, but do not remove transaction-ordering effects. Inclusion delays, fee changes before settlement, oracle observations, and revoke/close ordering can affect outcomes. Dynamo is not MEV-free merely because it uses signatures.

Oracles. Chainlink motivates authenticated external price inputs [6]. Dynamo's adapter checks freshness and deviation relative to a stored baseline. These checks constrain accepted observations; they do not independently establish a correct market price, eliminate a compromised feed, or guarantee that collateral remains sufficient after a price move (Section 5.9).

Protocol utility. A protocol token can support defined authorization, accountability, or execution functions. These are distinct from the asset used to pay a service invoice. Section 9 examines the proposed DYNAMO roles and their implementation requirements, including when an existing chain and ordinary collateral may be sufficient.

Table 1. Comparison of documented functions, not an exhaustive market survey.

System	Principal scope	Spending control and evidence	Distinction relevant to Dynamo
x402 [23-26]	HTTP payment negotiation and schemes	Exact and capped variable payments; batch settlement; signed offers/receipts	A recursive aggregate-budget policy is a separate composition question, not an absence of payment controls
MPP [18]	Machine payments across methods, including sessions	Method-specific authorization and settlement; evolving session tooling	Compare a named session method and implementation, not only bilateral request syntax
AP2 v0.2 [27]	Agent authority and transaction evidence	Constrained mandates, checkout/payment receipts	Authorization evidence is complementary to metered performance and funding state
ERC-8004 [12, 13]	Identity, reputation, validation	Registry-based claims; deployment-specific verification quality	Does not by itself provide Dynamo's budget conservation rule
Dynamo	Budget-tree metering and settlement	Contract accounting caps; cumulative vouchers; additional engine policies	Directly implemented escrow rules are narrower than the complete verified-value design

3. Design Doctrines

Six doctrines guide the architecture. Each specifies a design constraint together with the boundary at which it can be enforced.

D1 – Non-custodial operation. The protocol service is designed not to operate an omnibus user balance or perform fiat/crypto conversion. The vault administrator has no arbitrary principal-withdrawal method, but may change the fee within a 10% bound and change its collector. The budget owner has broader rights, including signing vouchers and closing budgets. A hosted operator may control that owner's key; an application treasury may receive end-user payments; a card payment service provider (PSP) controls parts of the payment process. Contract restrictions, key custody, application custody, and legal custody are therefore separate analyses. The four constraints are no arbitrary administrative withdrawal, redirection, freezing, or general stored-value account. They do not remove external token-issuer powers or make an operator-held key self-custody [32, 33].

D2 – Fail closed wherever money moves. Invalid or unverifiable authorization should prevent a new financial commitment. This is a safety policy, not a promise of availability or recovery. A rejected transaction does not undo work already performed upstream, reverse a previously finalized transfer, or automatically reconcile a partially completed cross-system operation. Fail-closed behavior must be specified per transition, including crash boundaries and retries.

D3 – Objective facts only. Settlement policies may use measurable quantities such as durations, unit counts, error rates, availability, or an explicitly recorded acceptance decision. A signature proves that a key attested to a statement; it does not prove the statement true. Percentiles require a specified population, sampling method, window, and clock. Explicit acceptance records a decision, not necessarily objective product quality. Subjective quality scores are excluded from the settlement kernel; richer quality markets require a separately specified validation layer.

D4 – Payload neutrality. The kernel is independent of prompts, media, or service content. Adapters map service measurements into canonical accounting units. They remain responsible for pricing correctness, measurement provenance, and any delivery predicate. A payload-free financial record can still reveal commercially sensitive metadata.

D5 – Mode identity. Budgets, streams, and cumulative claims share an interface across funding backends. Identity means an intended common object model and conformance obligations, not identical economic outcomes. A no-money ledger has no solvency risk; an ERC-20 transfer has chain-specific finality; a card authorization can expire and captures can be disputed. These differences must remain explicit (Section 4.2).

D6 – The data boundary. The metering interface is designed to transmit settlement facts rather than service payloads. Offline credential verification can avoid a remote verification request in the gateway path; self-hosting and opt-in telemetry support this boundary. Identifiers, amounts, timing, public-chain linkability, logs, and compromised adapters remain privacy risks even when prompts and outputs do not enter the financial record. They are part of the threat model [35].

4. System Model and Architecture

Implementation and evidence scope. Contract statements in Sections 4-6 refer to the immutable source snapshot listed in Appendix C. Proprietary engine, hosted-runtime, and card-backend details describe the project-reported reference implementation; they are not independent observations of every live instance. A later deployment must identify its code, configuration, keys, and resolved limitations before inheriting these claims. Interface documentation and public implementation files have different evidentiary weight.

4.1 Parties and layers

The parties are a **budget owner** authorizing expenditure, **service nodes** receiving settlement, an optional **oracle role** attesting breakpoints, a **fee collector**, and an **operator** running a core instance. An end user, an application treasury, the budget owner, and the protocol administrator need not be the same entity.

L0 provides funding, authorization, and rail-specific settlement. **L2** is the stream engine providing budgeting, delegation, metering, signing, and checkpointing. These are component labels; the present architecture does not require a separate Dynamo L1. The backend abstraction includes funding, funded-ceiling queries, settlement, refunds, and release. It isolates implementation concerns while preserving rail-specific risk.

Enforcement occurs at two points. The engine checks prospective accounting changes; the escrow contract independently checks the subset of rules encoded in its settlement function. The latter includes authenticated voucher authority, monotonicity, time bounds, the stream's rate envelope, ancestor caps, the budget cap, and sufficient asset balance. It does **not** evaluate every engine anomaly, SLA, exposure, or concurrency predicate. Re-implementation creates defense in depth and a need for differential testing; it does not make divergence logically impossible [32].

4.2 Three funding modes, one object graph

Mode	Ceiling	Settlement operation	Distinct assumptions
Control (`none`)	Local accounting allocation	Durable ledger entry, no value transfer	A simulated funding constraint; no proof of money backing
Escrow (`usdc`)	Deposit in the configured vault, with an accounting cap	Accepted voucher triggers ERC-20 transfers	Token behavior, chain execution, keys, gas, and collateral valuation
Card (`card`)	PSP authorization for a bounded amount	Captures and connected-account transfers	PSP capability/approval, authorization lifetime, cent granularity, disputes

Changing the funding selector can preserve application-level object shapes, but production migration also requires keys, asset and chain configuration, PSP onboarding where applicable, and error/finality handling. The published client, `@dynamoprotocol/sdk` 2.1.0, is a thin HTTP interface rather than an embedded engine. Engine capabilities must be distinguished from operations exposed by that client [35, 38].

Conformance criterion. For a common supported operation, compare normalized preconditions, accounting state transitions, and error categories; separately document permissible rail differences. A successful Control Mode test is useful but cannot validate an ERC-20 transfer or a PSP's production acceptance. Reusing a schema is not an observational-equivalence proof.

Payment timing vocabulary. We distinguish five milestones: service delivery; acceptance of a signed payment claim; successful backend settlement; the deployment's chosen finality threshold; and funds usable by the provider, including any necessary conversion or withdrawal. "Streamed by the second" describes the metering and payment model, not a requirement for one ledger transaction every second. "Faster payment" is evaluated using the distribution of elapsed time from eligible delivery to usable funds, together with the fraction of delivered work never collected. A card capture or internal transfer is not necessarily a completed bank payout.

4.3 Threat model (summary)

Adversaries may control agents or service nodes; obtain a scoped credential; replay or reorder messages; withhold a result; collude with an attester; or exploit delayed observations. Hosts may crash, clocks may disagree, networks may partition, chains may reorganize, and a PSP may decline or time out. Administrators, budget-owner signers, application treasuries, token issuers, and measurement sources have distinct powers and cannot be merged into one generic trusted role.

The contract results in Section 5 assume: (A1) atomic execution and rollback of the modeled Ethereum Virtual Machine (EVM) transitions; (A2) unforgeable signatures and correctly bound keys/domains, except for explicitly compromised authorized keys; (A3) the non-upgradeable contract behavior at the cited source snapshot and valid arithmetic ranges; (A4) settlement assets satisfying the modeled transfer semantics, without unmodeled rebases or balance seizures; and (A5) observations of the relevant canonical chain state. End-to-end spending control additionally requires (A6) non-bypassable admission and atomic reservations or disjoint preallocated authority. Provider payment and availability require further liveness assumptions and are not implied by A1-A6.

4.4 The request path and durability

Two request paths must be distinguished. In the **owner-API path**, the HTTP SDK calls the core; the engine checks a metering request, records it, signs a voucher when appropriate, and checkpoints through the backend. This path incurs network and processing latency. In the **gateway path**, allowance credentials can be verified locally before forwarding service traffic, with subsequent metering reports. Local verification removes a particular remote call, not all computational overhead, and does not by itself synchronize a shared budget across replicas [35].

The engine uses file-backed SQLite write-ahead log (WAL) journaling, `synchronous = FULL`, and a pinned schema version. Recovery discards consume records not covered by a signed voucher and releases held reservations, establishing a local billing frontier. These mechanisms do not make a transaction spanning SQLite, an EVM chain, and a PSP atomic. Backend success can precede the local record write, while the revocation path commits locally before calling the backend. The hosted runtime also has a budget-rehydration limitation after restart. A complete recovery procedure must address each of these boundaries rather than relying only on database durability.

Thus, “billing truth is the last signed voucher” means the intended durable local entitlement frontier, not a complete cross-system recovery algorithm. An independently observed backend transfer remains a financial fact even when its checkpoint row is missing. A production recovery design needs durable operation identifiers, queryable backend outcomes, idempotent retry, and reconciliation before reopening spending authority. These are acceptance requirements where the current implementation does not yet supply them.

4.5 The accounts model

The escrow deployment has five operational roles: the **deployer** owns initial administration; the **core owner** funds and closes budgets; the **service node** submits settlements and needs gas; the **application treasury** collects end-user payments and issues user refunds; and the **fee collector** receives fees. Roles may share addresses, but sharing changes compromise boundaries. Monitoring must distinguish token balances from gas balances.

For a job settling gross accounting amount D in a six-decimal identity-priced asset, the core owner funds the escrow and receives only its unspent remainder; the service node receives D less the fee; the collector receives the fee. An application that collected the user’s payment holds a separate obligation. Closing the vault returns principal to the recorded budget owner, **not automatically to the application’s end user**. The application’s refund is a separate transfer, and its solvency and reliable outbox are separate responsibilities [32, `closeBudget`].

Combining budget-owner and payee addresses can reduce treasury drift in a self-operated test, but also combines their compromise boundaries. Such a configuration does not demonstrate settlement between independent counterparties. Evaluation must identify which roles are independently controlled, and applications must monitor the funding account separately from the account that collects end-user payments.

5. The Core Protocol, Formally

5.1 The budget tree

Let a budget B have accounting cap $C \in \mathbb{N}_{>0}$. Amounts are nonnegative integers in micro-USD accounting units, with 10^6 units per USD denomination. This is an accounting convention; a token’s actual purchasing power may differ. All divisions in the financial kernel require a specified rounding direction. The implementation uses integer arithmetic; overflow must reject rather than wrap [32, 33].

A stream is represented by

$$s = (\mathrm{node}_s, r_s, c_s, t_0(s), \rho_s, d_s), \qquad (1)$$

where $r_s > 0$ is an accounting-unit-per-second envelope, $c_s > 0$ is a subtree cap, $t_0(s)$ is the backend start time, ρ_s is a revocation timestamp or $\perp$, and $1 \le d_s \le 8$. A budget has a forest of root streams; adjoining a virtual budget root gives a tree. A service dependency graph with joins or cycles is not literally this tree: its obligations need an unambiguous allocation mapping to avoid double counting.

Opening a root requires $\sum_{i \in \mathrm{roots}(B)} c_i \le C$. Opening a child of p requires

$$c_s + \sum_{j \in \mathrm{existingChildren}(p)} c_j \le c_p. \qquad (2)$$

Allocations are monotone: revocation does not return the allocated cap. This avoids reusing authority that may still be associated with outstanding claims, but strands capacity and can exhaust allocatable headroom without exhausting funds. It is one conservative implementation choice, not the only safe allocation design. A child

allocation also is not a guarantee that its full cap remains spendable: the parent's own payments share the same subtree cap [32, `_openStream`, `settle`].

5.2 The cap invariant

Write S_s for stream s 's **gross cumulative settled accounting amount**, $\mathcal{T}(s)$ for its subtree, and

$$Q_s = \sum_{u \in \mathcal{T}(s)} S_u, \qquad T_B = \sum_{u \in B} S_u. \qquad (3)$$

Refunds do not decrement these gross counters in the contract. Accordingly, they do not restore consumed cap capacity. The relevant safety statement is

$$\forall s: \quad 0 \le Q_s \le c_s, \qquad 0 \le T_B \le C. \qquad (4)$$

Proposition 1 (accounting-cap conservation). Under A1, A3, and the specified transitions, every reachable successful-settlement state satisfies (4), irrespective of transaction ordering or whether an authorized voucher signer is honest.

Proof. Initially all settlement counters are zero. Opening a stream creates zero counters and checks allocation rule (2). Consider settlement of a voucher with cumulative amount $V > S_s$, and define $\delta = V - S_s > 0$. The contract adds δ to each subtree counter on the path from s to its root and rejects if any updated counter exceeds its cap. It similarly requires $T_B + \delta \le C$. Counters outside that path are unchanged. If a later conversion or transfer fails, atomic rollback restores the previous state. Revocation, attestation, refund, and closure do not increase the gross settlement counters. Induction over successful transitions establishes (4). □

The proposition bounds **gross settlement in accounting units**. It does not bound gas paid separately, external provider invoices, card disputes, or the USD value of a volatile token at all future prices. In a six-decimal identity-priced asset, gross voucher settlement includes both payee and protocol fee. With oracle-priced assets, accounting conservation and asset-balance conservation are distinct statements.

Let E_B be current escrow in asset base units, D_B cumulative deposits, R_B successful refund inflows, P_B settlement outflows, and L_B closure releases. Under A4 and excluding unsolicited transfers from the per-budget accounting, the corresponding flow identity is

$$E_B = D_B + R_B - P_B - L_B \ge 0. \qquad (5)$$

For multiple budgets sharing one token balance, the accounting must additionally reconcile the sum of per-budget escrow records with the vault's actual token holdings. External token behavior can invalidate that reconciliation even though the accounting inequalities remain true.

Engine admission. An analogous engine invariant must count accrued usage and held reservations on every ancestor before admitting new obligations. Atomicity of the test-and-update step is essential. The contract's proof cannot be reused as a proof of pre-execution control without establishing that engine invariant and the integration boundary in A6.

5.3 The rate envelope

For an accepted escrow voucher with declared timestamp τ , the contract requires

$$V \le r_s \left[\min(\tau, t_b) - t_0(s)\right], \qquad (6)$$

when an attested breakpoint t_b exists; otherwise use τ as the endpoint. It also requires $t_0(s) \le \tau \le t_{\text{block}}$ and, after revocation, $\tau < \rho_s$. Without a breakpoint, the familiar engine-side form is

$$A_s(t) + H_s(t) \le r_s \max(0, t - t_0(s)), \qquad (7)$$

where A_s denotes own accrued accounting units and H_s held own reservations. The engine uses its clock, whereas escrow start time is chain-derived. Clock agreement is an operational condition, not a consequence of integer arithmetic [32].

Corollary 1 (no first-second grace). At $t = t_0(s)$ the envelope is zero. A positive cumulative charge W first fits at an integer elapsed time of at least $\lceil W/r_s \rceil$ seconds. Waiting one second suffices only when $W \le r_s$ and the relevant chain/host clocks have advanced appropriately.

Corollary 2 (fastest-profile sizing). To admit a planned cumulative workload W_k by elapsed time $\tau_k > 0$, choose

$$r_s \ge \max_k \left\lceil \frac{W_k}{\tau_k} \right\rceil. \qquad (8)$$

A bound based on average completion time can reject a faster realization. This is a consequence of the authorization model, not evidence that fast work is anomalous. Burst allowances or alternative envelopes would be distinct protocol changes.

Corollary 3 (bankable idle headroom). Because the bound is cumulative from stream opening, idle time creates later headroom, still limited by the cap. It does not bound expenditure in every sliding time window. A long-idle stream may settle a large burst. This matters both for benign bursty workloads and for the damage possible after key compromise.

Timestamp limitation. A signed `signedAt` field authenticates a signer's declaration, not the actual signing time. An authorized compromised signer can attempt to backdate a voucher within the admitted window. The current checks constrain the declared-time envelope and total cap; they do not prove that all work or all signature creation occurred before revocation. Stronger temporal guarantees require a trusted measurement/timestamp mechanism, prior commitments, or a different revocation rule.

5.4 The enforcement predicate

The engine's enforcement precedence is the first applicable reason in

$$\mathcal{E}(s, \Delta) = (\text{revoked}, \text{anomaly}, \text{sla}, \text{budget_closed}, \text{exposure}, \text{rate}, \text{payer_concurrency}, \text{cap}). \qquad (9)$$

This is a priority-ordered decision procedure, not an algebraic proof that each predicate is monotone. For fixed state, configuration, clock, and observations, a deterministic implementation should return the same verdict. Across time, a rate refusal can resolve as the envelope grows; an exposure refusal can resolve after settlement; and a concurrency refusal can resolve after a reservation closes. Revocation and budget closure are terminal in the modeled lifecycle; an anomaly/SLA halt is sticky only when the appropriate persistent marker is written.

Anomaly persistence is path-dependent: a halt is terminal only when its persistent marker is written. The published HTTP SDK does not expose anomaly-envelope configuration. The runtime's decision sequence must also be tested against the declared precedence, rather than assuming that an exported precedence constant is the executed policy. A temporary refusal therefore cannot be interpreted as a permanent stream halt.

Nor is the full list enforced by `BudgetVault.settle`. The contract enforces its own narrower predicate. Agreement for shared predicates is a conformance obligation; disagreement may fail safely at settlement, strand work, or create an operational incident. A rejected settlement is not evidence that the two layers never diverged.

5.5 Monotone vouchers: ordering replaces nonces

A settlement voucher has the EIP-712 typed structure

$$v_k = (\text{streamId}, U_k, V_k, \tau_k), \qquad (10)$$

with cumulative meter reading U_k, cumulative accounting amount V_k, and declared time τ_k. The contract requires $V_k > S_{k-1}$ and a nondecreasing meter reading; it does not derive the amount from the meter or validate an external tariff. The engine convention sets them numerically equal, but the contract's amount authority is the signed amount field [32].

Settlement pays gross accounting delta

$$\delta_k = V_k - S_{k-1}, \qquad S_k = V_k. \qquad (11)$$

Proposition 2 (no duplicate gross settlement). For any accepted sequence against one stream's persistent high-water state,

$$\sum_{k=1}^{n} \delta_k = S_n - S_0. \qquad (12)$$

Proof. Substitute $\delta_k = S_k - S_{k-1}$ and telescope. A duplicate or lower cumulative value fails the strict-advance check; a higher value pays only the increment not already settled. EVM rollback preserves the previous high-water state on failure. □

The result is independent of which intermediate vouchers are omitted or presented out of order, provided the accepted cumulative amounts advance. It establishes economic at-most-once settlement, not guaranteed submission, fair payment, delivery truth, or exactly-once messaging.

EIP-712 domains bind typed signatures to a name, version, chain identifier, and verifying contract [4]. The escrow domain uses the actual chain and vault. Off-chain backends use synthetic domains; their instance isolation also depends on correct key and stream bindings. Domain separation alone is not a substitute for those bindings. Allowance credentials use separate nonces and revocation records, and the public SDK exports nonce-related functionality. Cumulative settlement therefore requires **no separate per-voucher nonce**, while credential nonces retain their own role [35].

Revocation invalidates vouchers whose declared timestamp is at or after the recorded revoke time. Earlier-dated vouchers may still settle until the budget closes. The operator's graceful sequence is to checkpoint accepted work, verify the outcome, and then revoke. This is especially important because same-second vouchers are rejected and closure provides no payee challenge window.

5.6 Idempotent metering and two-phase reservations

A metering request carries a caller-selected `requestRef`, unique within its stream: printable ASCII of length 1-128. Returning the prior result for a repeated reference provides retry tolerance; checking for duplicates before new admission checks lets an already billed request be acknowledged after a halt. This relies on a durable uniqueness constraint and correct record-status handling. A recovery edge case can return a stale cumulative result from a discarded record. Crash-safe retry semantics therefore require testing both uniqueness and status, not merely equality of request identifiers.

Delivery-gated application billing can use a stable delivery identifier and bill only after success. This prevents a failed delivery from being billed **only if** the adapter's success criterion is correct and no earlier irreversible charge was already made. An upstream provider may charge for unsuccessful or canceled work under its own terms. Dynamo's application receipt and the provider's invoice must not be treated as the same obligation.

For uncertain costs, the embedded engine offers reservation-style operations: reserve an upper-bound amount, execute work, then confirm within the reservation or cancel unused authority. Capacity is counted along the ancestor path when reserved; conversion to billed usage must not count the same capacity twice. However, revocation, budget state, authorization freshness, and policy changes require an explicit confirm-time rule. Avoiding double counting does not justify bypassing every safety check.

Reservation consumption rechecks only part of the ordinary enforcement ladder. The published HTTP SDK does not expose `reserve`/`confirm`/`cancel` as public methods. These operations are engine mechanisms; using them for pre-execution control requires an integration that exposes them and defines confirm-time policy checks. Their existence in the engine does not imply availability through the public client.

Concurrent admission requirement. A headroom read followed by an uncoordinated provider call is insufficient: two workers can each observe the same remaining amount and both spend it. A safe implementation must atomically reserve from shared state, allocate disjoint sub-budgets/leases, or enforce an equivalent serialization rule. Offline verification alone does not solve this shared-state problem.

Admission lemma for an integration. Let a_s be own irrevocably incurred accounting obligations, including those already settled, and let h_s be outstanding reserved upper bounds. Define subtree committed authority as $P_s = \sum_{u \in \mathcal{T}(s)} (a_u + h_u)$. If initially $P_s \le c_s$ and the analogous budget bound holds, an atomic reservation of x is safe when it checks every affected ancestor and the budget before adding x to h_s. Confirmation replaces a reserved amount x with an actual obligation $y \le x$; cancellation removes x only once no associated payable work can still occur. Each operation either checks the bound before increasing it or decreases it, proving conservation by induction. Settling an existing obligation must not add it a second time.

This lemma specifies the integration requirement rather than proving that every exposed Dynamo path implements it. It assumes an enforceable upper bound on cost, atomic state transitions, and complete accounting of billable work. Cancellation cannot safely free capacity merely because a network response timed out. The claim does not apply to unknown-price commitments, bypassed provider calls, or disconnected workers sharing overlapping authority.

5.7 Fee arithmetic and the sustainability condition

Let $K = 10^4$ and let $f \in \{0, \dots, 1000\}$ be the escrow fee in basis points. For gross accounting delta δ, the contract computes

$$F_f(\delta) = \left\lfloor \frac{\delta f}{K} \right\rfloor, \qquad N_f(\delta) = \delta - F_f(\delta). \qquad (13)$$

For a six-decimal identity-priced asset, those values are the collector's and payee's asset amounts. For an oracle-priced asset, each is converted separately and rounded down. The fee is the rate in force **at settlement**, not necessarily the rate when the stream was opened. The administrator's ability to change it within its bound must therefore be disclosed to payers and payees [32, `_payOut`, `setFeeBps`].

Proposition 3 (zero-fee threshold and rounding loss). If $f = 0$, all protocol fees are zero. If $f > 0$, then

$$F_f(\delta) = 0 \iff \delta < \frac{K}{f} \iff \delta \le \left\lceil \frac{K}{f} \right\rceil - 1. \qquad (14)$$

For $n > 0$ settlements totaling $D = \sum_i \delta_i$, revenue $R = \sum_i F_f(\delta_i)$ satisfies

$$0 \le \frac{Df}{K} - R < n, \qquad R \le \left\lfloor \frac{Df}{K} \right\rfloor. \qquad (15)$$

Proof. The first equivalence is the definition of a floor equaling zero for a nonnegative argument. Apply $x - 1 < \lfloor x \rfloor \le x$ to each fee and sum. □

At the documented deployment default of 100 bps, a delta below 100 micro-USD produces zero **protocol** fee; gas is not thereby free. The floor loss can matter if settlement is highly fragmented, and a payee controlling checkpoint granularity may reduce fees by splitting claims. Batching, a minimum settlement size, or cumulative rather than per-delta fee assessment can change this incentive. The last alternative would be a protocol change, not a description of the contract.

Sustainability condition. Let G be the session's modeled settlement cost, converted into the same accounting denomination as revenue. The inequality

$$\sum_i \left\lfloor \frac{\delta_i f}{K} \right\rfloor \ge G \qquad (16)$$

is exactly a *settlement-cost coverage condition*. It is necessary for a model in which those fees alone pay those costs, and sufficient only for that restricted model. Business sustainability additionally depends on who pays gas, operating and support costs, audit costs, losses, taxes, subsidies, and revenue retention. A more complete contribution-margin condition is $R \ge G + O + L$, with overhead allocation O and expected loss L. A fee directed to a collector does not reimburse a gas-paying service node automatically.

The fee documentation describes a harness using recorded session shapes and PSP TEST-mode activity [40]. Such fixtures evaluate the specified cost model rather than establish profitability for every workload. On the card rail, PSP charges and a separate application fee, documented as 250 bps by default, must be modeled independently, including non-returned processing charges on refunded value. The verified-layer fee ledger, the card fee, and the vault fee are distinct mechanisms. The verified-layer fee is zero on the hosted SDK path and must not be confused with the on-chain vault fee.

Gross-to-gross reconciliation. At a completely drained, successfully reconciled billing frontier, in the identity-priced escrow mode,

$$\text{gross billed} = \text{gross engine settled} = \text{gross vault settled}. \qquad (17)$$

Before that frontier, billed but unsettled claims can make the first equality false. Refunds, fees, pending claims, and asset conversion require separate columns; they must not be silently netted into `settledTotal`. On-chain payee receipts plus fee receipts equal the asset outflow, while oracle rounding can prevent a one-for-one comparison between asset base units and micro-USD accounting units.

5.8 Verified-value settlement

Verified-value settlement associates a stream with canonicalized service-level agreement (SLA) terms, signed observations, and a breach policy. Strict schemas admit specified integer-valued or decimal-string measurements;

observation gaps and invalid signatures are recorded rather than treated as successful evidence. Consecutive breached windows can progress through `breach-observed`, `throttle`, and a persistent `halt`; an adequate non-breached window resets the consecutive-breach counter.

To make the policy testable, an SLA must identify the measured event, unit and tariff, observation window, minimum sample size, clock and percentile estimator, signer authority, maximum observation age, duplicate policy, and treatment of missing observations. An error-rate threshold without a denominator or a latency percentile without a window is not a complete settlement rule. The observation source may still be dishonest or collusive; a cryptographic signature does not remove that assumption.

Implementation boundary. The vault stores `slaTermsHash` but does not interpret it. It does not directly enforce a latency percentile or determine whether a video, model response, or game is correct. Anomaly configuration is not exposed by the published HTTP SDK, and higher-level runtime scheduling remains incomplete. Authenticated measurement, engine policy, and contract settlement are separate enforcement surfaces; the complete objective-fact design is not an unconditional end-to-end property of the public SDK and escrow combination [32].

Breakpoint refunds. For a stream with settled amount S_s and attested breakpoint t_b, the implemented target is

$$\Phi_s = \max\{0, S_s - r_s(t_b - t_0(s))\}, \qquad \Delta\Phi_s = \Phi_s - \Phi_s^{\text{already}}. \qquad (18)$$

A call executes only when the target exceeds the stored refund high-water mark. This prevents duplicate economic refunds; repeated calls may revert instead of returning a successful idempotent acknowledgment. Gross settled and allocated-cap counters are not reduced.

Proposition 4 (scope of the refund target). Equation (18) removes settlement exceeding the maximum amount authorized by the pre-breakpoint envelope. It is not generally the exact amount attributable to work delivered after the breakpoint.

Justification and counterexample. Let $r_s = 100$, $t_0 = 0$, and $t_b = 2$. Suppose no work was delivered before t_b, and 100 units were settled later before the breakpoint was attested. Equation (18) gives $\Phi_s = \max(0, 100 - 200) = 0$, even though all 100 delivered units were post-breakpoint. A rate envelope is an upper bound, not a record of when work occurred. Recovering the exact post-breakpoint entitlement requires authenticated cumulative earned value at the breakpoint, or equivalent per-unit evidence, rather than this envelope alone. □

Funding of refunds: implemented versus proposed. In the vault, successful settlements transfer funds immediately. `refundAfterBreakpoint` pulls the refund from the service node with ERC-20 `transferFrom`; it requires the payee's approval and sufficient transferable balance. The refund is gross, including the earlier protocol-fee component, while the fee collector is not debited. It is therefore neither a solvency-independent refund nor a refund automatically paid from a funded holdback reserve [32, `refundAfterBreakpoint`].

A proposed **buyer-whole-first** mechanism would retain eligible contingent value in enforceable escrow or collateral until an agreed dispute window ends, then pay eligible refunds from that reserve before distributions. A bookkeeping finality delay alone cannot supply the reserve. Its amount, control, priority, liquidation or transfer rules, and exhaustion behavior require explicit specification and testing. After irrevocable release, recourse is limited to remaining collateral and applicable counterparty obligations. This reserve-backed mechanism is distinct from the implemented payee-funded refund path.

5.9 Oracle-priced assets and the round-trip lemma

Let a be token base units, d the **token's** decimals, $p > 0$ a price normalized to micro-USD accounting units per whole token, $K = 10^4$, and $k = K - h_o$, where haircut $0 \le h_o < K$. The registry implements

$$T_p(a) = \left\lfloor \frac{k}{K} \left\lfloor \frac{ap}{10^d} \right\rfloor \right\rfloor, \qquad A_p(x) = \left\lfloor \frac{x\,10^d K}{pk} \right\rfloor. \qquad (19)$$

The inverse divides by the haircut-adjusted price. It is not a second haircut multiplication. Feed decimals are normalized by the adapter; confusing them with token decimals changes the economic scale [33].

Proposition 5 (fixed-quote round-trip non-inflation). For one fixed valid price, haircut, and decimal configuration, and arithmetic within implementation bounds,

$$A_p(T_p(a)) \le a. \qquad (20)$$

Proof. Since $T_p(a) \le apk/(10^d K)$, multiplying by the positive factor $10^d K/(pk)$ yields at most a. Flooring that quantity cannot increase it. □

The result does not require each conversion to be individually non-expanding: the two conversions use different units and one factor can exceed one. It also does not hold across arbitrary quote changes. If the price halves between conversion into accounting units and conversion back, the same accounting claim can correspond to approximately twice as many tokens. A haircut is a collateral buffer, not a proof against unlimited depreciation or oracle manipulation.

For payout splitting at one fixed quote, $A_p(x) + A_p(y) \le A_p(x + y)$; the difference is less than two token base units and, for two terms, at most one. Remaining dust stays in the vault accounting until release. Asset decimals, checked multiplication ranges, minimum payable amounts, and rounding residues belong in conformance tests.

Fail-closed pricing. Registry conversions reject unlisted assets, zero prices, zero/future timestamps, and stale observations. The separate adapter applies a deviation band and an observation baseline, with a permissionless baseline-update operation. This is a bounded-change check, not a defense against all economically harmful drift [33]. `closeBudget` does not consult the oracle, so stale prices alone do not block its conversion-free release path. Closure still requires all streams revoked, sufficient gas, functioning token transfers, and an executable chain.

Stable listings are exactly six-decimal and identity-priced. That is a declared peg assumption; the registry does not test real-world peg stability. Add-only asset/oracle configuration avoids retroactive administrator edits but also prevents correction of a bad listing in that deployed registry.

5.10 Card-rail epoch settlement

The card backend uses a manual-capture authorization rather than a stored protocol balance. Funding rounds an accounting cap upward into cents:

$$A_{\text{cents}} = \left\lceil C/10^4 \right\rceil. \qquad (21)$$

The cap on accepted accounting usage remains C; the rounded authorization need not be fully captured. Capture strategy is capability-dependent: `epochCapture` requires multicapture support, `finalCapture` permits one final capture, and `auto` selects between them. The implementation is restricted to **Stripe TEST mode**; a tested code path is not a production payment-service approval [30].

Let chargeable accounting amount be $Q = \sum_s (S_s - R_s^{\text{accounting}})$. With gross captures X and PSP refunds Y, both in cents, the documented capture-debt model is

$$Z = \lfloor Q/10^4 \rfloor, \qquad X_{\text{net}} = X - Y, \qquad \Delta_{\text{cap}} = Z - X_{\text{net}}. \qquad (22)$$

A positive debt can be captured only when the strategy and remaining authorization permit it. A nonpositive debt is not a negative capture; it belongs to the refund/reconciliation path. The governing safety constraint is

$$\sum \text{gross captures on an authorization} \le A_{\text{cents}}. \qquad (23)$$

Refunded value does not restore the same authorization's capture capacity in this model. Re-earned value after a refund can therefore require a new authorized ceiling. The backend's renewal decline is a state result (`renewed: false`, with decline detail) that marks the ceiling dead; it must not be ignored by callers that handle only exceptions.

Payout targets floor to cents. Sub-cent residues are recorded at release, but recording a liability or residue is not equivalent to paying it. The residual ownership, eventual aggregation, and disclosure policy must be explicit. Card reconciliation can tolerate a sub-cent shortfall under the specified model, while over-capture is a failure; this is a rail-specific tolerance, not identity with escrow arithmetic.

Authorization expiry, multiple-capture eligibility, transfer reversal, chargebacks, and PSP outages constrain liveness. The backend's modeled seven-day authorization TTL is not a universal rule about every card authorization. A production adapter must use the actual supported authorization and capture rules and distinguish API acceptance, capture, transfer, payout, and dispute finality [30].

5.11 Subscriptions as recurrence

The subscription design is a recurrence wrapper: a fresh period creates a new funded budget from a declared template. On cards, each period requires an authorization rather than interpreting card-on-file possession as unlimited spending authority. The template specifies whether unused ceiling is released or contributes to the next period's cap. Rollover of a *limit* must not be confused with automatic transfer of money or revival of an expired authorization.

Renewal failure stops new unfunded accrual; previously funded obligations retain their separately defined settlement rights. Period-open, renewal-decline, and close events provide a versioned integration interface. Consumers still need durable deduplication and a recovery/query mechanism; webhooks alone do not guarantee delivery.

Subscriptions and some batch operations are engine capabilities rather than fully exposed public HTTP-client methods [38]. Production subscription support requires a defined wire interface, an end-to-end authorization lifecycle, and consent and compliance review. The recurrence model does not by itself establish those operational capabilities.

5.12 Exposure ladders and the timing trade-off

The engine model caps unsettled service-node exposure at a configurable $E_{\max}$, defaulting to USD 1,000 in accounting units. An exposure bound must identify its scope–instance, budget, service node, or combination–and include pending reservations. A bound of $E_{\max}$ plus one in-flight request applies only to serialized admission with one such request; with multiple unreserved concurrent requests, the additional liability is their sum. These conditions determine the actual bound on provider credit.

The payer-side degradation ladder uses remaining budget headroom H_B and

$$q = \lfloor 100 \max(0, H_B)/C \rfloor . \qquad (24)$$

The reported defaults are: at least 50% gives `full`; 25-49% gives `throttled`; 10-24% gives `checkpoint-required`; below 10% gives `refuse`. Throttling constrains request size, and checkpoint-required blocks additional accrual until outstanding claims settle. These are conservative policy thresholds, not unavoidable consequences of a funded cap. In particular, a deliberate refusal may leave budget unused.

Checkpoint cadence. Let $c > 0$ be an interval, $g > 0$ cost per checkpoint, $r > 0$ a constant admitted accrual rate, and $\lambda > 0$ a risk-price coefficient with units chosen so that both terms below are cost per unit time. A peak-exposure approximation gives

$$J(c) = \frac{g}{c} + \lambda r c, \qquad c^* = \sqrt{\frac{g}{\lambda r}}, \qquad J(c^*) = 2\sqrt{g\lambda r}. \qquad (25)$$

Derivation. $J'(c) = -g/c^2 + \lambda r$ vanishes at c^* ; $J''(c) = 2g/c^3 > 0$, so this is the unique unconstrained minimum. If the risk term instead prices the time-average exposure of linear accrual, the average is $rc/2$ and the optimum is $\sqrt{2g/(\lambda r)}$. Choosing between them is a modeling decision.

A deployment must also impose feasible bounds: maximum allowed latency, expiration, risk ceilings, block inclusion variability, and batching rules. The constrained optimum is the unconstrained value clipped to a feasible interval when that interval is nonempty. At zero risk price or zero accrual, no finite interior optimum follows from (25); at zero checkpoint cost the model favors the smallest feasible interval. Bursty workload and variable gas costs require a different objective or sensitivity analysis.

Holdback. A proposed holdback h_b makes claims conditionally final until a declared release criterion, but the number of checkpoints within a trailing window depends on phase and can differ by one from $\lfloor h_b/c \rfloor$. More importantly, a time label does not imply refund-reachable funds. The vault pays immediately, and the hosted runtime does not schedule the finality polling required by a nonzero holdback configuration. Setting `holdbackSeconds: 0` addresses that scheduling limitation; it does not make chain finality immediate or create a solvency-independent refund reserve [41].

6. Security Properties and Analysis

6.1 Scoped credentials and bounded blast radius

A correctly provisioned agent receives narrowly scoped authority rather than the budget owner's unrestricted key. For a voucher signer authorized on only one stream, the contract limits gross accepted settlement to the minimum remaining headroom across that stream's ancestor path and its budget. The rate envelope adds a declared-time constraint. Foreign-stream rejection additionally depends on unique or correctly bound session keys; reusing the same authorized key across streams changes the compromise scope [32].

This bound does not apply unchanged to a leaked owner API token, budget-owner key, treasury key, or PSP credential. The vault accepts its budget owner as a fallback voucher signer across every stream in that budget. The hosted core also handles signing material and can be a substantial trust boundary. Local compromise may affect confidentiality and future authorized expenditure even when cap conservation survives.

Allowance-credential revocation requires the enforcing gateway to learn the relevant revoked nonce or updated policy. An offline verifier with a stale revocation view cannot know a remote revocation instantaneously. Short

lifetimes, bounded refresh intervals, and disjoint spending leases can bound exposure; their parameters must appear in the security claim.

OWASP's taxonomy is useful for organizing threats, not certifying conformance. ASI01 concerns agent goal hijack, ASI02 tool misuse and exploitation, and ASI03 identity and privilege abuse [16]. Caps can reduce the financial consequences of compromised goals and tool use; they do not prevent prompt injection, make tools safe, or eliminate identity abuse.

6.2 Threat classes and their answers

Table 2. Defenses and residual conditions.

Threat	Supported defense	Residual condition or required evidence
Leaked scoped key	Stream/ancestor/budget accounting caps; signer checks	Key isolation, timestamp trust, and non-bypassable provider access
Replayed voucher	Persistent cumulative high-water state	Correct domains, instance bindings, canonical-chain observation
Stale or manipulated price	Freshness and deviation checks; payout rejects if underfunded	Feed trust, baseline drift, collateral loss, depeg and token-issuer powers
Runaway loop	Cap, cumulative envelope, configured admission policy	Idle headroom, concurrent work, external costs; SDK anomaly feature unavailable as documented
Crash during settlement	Durable local records and signed frontier	Cross-system recovery windows remain; reconcile actual chain/PSP outcomes
Revocation race	Reject declared timestamps at/after revoke	Same-second loss, backdating, delayed inclusion, gas limits, early close
Malicious administrator	No arbitrary vault-principal withdrawal; bounded fees	Owner/collector changes, fee timing, different roles' keys, token controls
Counterparty dispute	Signed and digest-checked records	Trusted identity anchors, truthful observations, completeness, applicable procedure

Revocation liveness. The implementation recursively writes every node in the revoked subtree. Depth is limited to eight, but breadth is not comparably bounded. A sufficiently large subtree can exceed a transaction's gas budget, so "one-call instant subtree revocation" requires a tested size limit and successful inclusion. Revoking smaller subtrees is a mitigation, not an unconditional atomic revocation guarantee for an arbitrary network [32, `_revokeSubtree`].

Payee liveness. After all streams are revoked, the owner may immediately close a budget. Unsubmitted otherwise valid vouchers then become unsettleable. This is an explicit counterparty risk. A guaranteed claim-submission window would require a different closing protocol. Finite checkpoint cadence reduces exposure but does not eliminate the race.

6.3 Fail closed as the system property

Malformed signatures, nonadvancing cumulative claims, and invalid conversion inputs cause relevant contract transitions to revert. That is a meaningful financial safety boundary. However, fail-closed operation can preserve principal while denying service or withholding legitimate payment. It cannot simultaneously guarantee instantaneous revocation, uninterrupted offline service, and globally shared uncoordinated spending authority.

A failure taxonomy should distinguish **rejection before an obligation**, **rejection after external work**, **unknown backend outcome**, **confirmed transfer with missing local acknowledgment**, and **irrecoverable application state**. Only the first directly supports a “no new cost” claim. Unknown outcomes must quarantine the affected authority until reconciliation; treating a timeout as proof of no transfer can double-pay or double-allocate.

The administrator surface is limited in the contract, but absence of an arbitrary withdrawal function is not administrator powerlessness: fee changes alter payee proceeds. Testing intended negative capabilities is useful evidence; it is not a proof about arbitrary future upgrades or every external dependency [32].

6.4 Evidence and dispute verification

Dynamo’s evidence design assembles session records, vouchers, credentials, attestations, settlement references, refund records, and a content-digest manifest in an `evidence/1.0.0` bundle. An offline verifier can check included signatures, recompute digests, and recompute accounting totals. The evidence documentation reports a 71-check offline demonstration and describes operator-mediated export rather than a supported public SDK export method [39]. The objective is dispute-oriented evidence with explicit verification assumptions.

Three boundaries are essential. First, verification material included in a bundle is not automatically trusted: replacing both a public key and all signed records can produce a self-consistent counterfeit unless an identity or chain root is independently pinned. Second, signature validity does not prove measurement truth, the completeness of the event set, or current chain/PSP finality. Third, an offline snapshot cannot learn subsequent revocation or a later reorganization without new authenticated information.

The per-call SDK receipt is a separate artifact: `dynamo-receipt-v1` is signed with EIP-191 over the serialized payload by a session key. Settlement vouchers use EIP-712, while allowance and fact credentials use JWS/ES256K. These formats have different purposes and verification rules. A per-call receipt is not a complete evidence export.

A reproducible evidence fixture needs a pinned trust root, canonicalization rules, expected success and failure outputs, a completeness boundary, and offline verification commands. A one-byte tamper test evaluates the authenticated envelope that contains that byte; a change outside the envelope need not have the same effect. Results must identify the covered bytes and checks. Compatibility with Visa TAP, Mastercard Verifiable Intent, or a card dispute process requires separate conformance evidence [28, 29].

6.5 Mechanism-level honesty

One-way cap allocation and **same-second clipping** are explicit conservative choices. The former trades allocation flexibility for simpler authority accounting. The latter declines ambiguous declared-time claims at the revocation boundary. Neither replaces a complete threat model: reserved caps can become unusable, and valid pre-revocation work can go unpaid if settlement is delayed.

The graceful public-client sequence is `checkpoint({force: true})`, verify completion, then `revoke()`. `force` bypasses the checkpoint-due policy, not signature, funding, monotonicity, or finality checks. If the backend revocation fails after local commitment, local state is not proof of chain success; the hosted retry path does not automatically re-drive that backend operation. The affected authority must remain stopped until its backend state is reconciled.

Mechanism-level honesty also requires identifying simulations, TEST-mode PSP calls, testnet value, and production transfers distinctly. A signed receipt and a realistic transaction hash-shaped string are not interchangeable. An unavailable reconciliation result is **unverified**, not automatically reconciled and not necessarily a mismatch. These distinctions are part of the protocol’s auditability rather than cosmetic qualifications.

7. Implementation and Evaluation

7.1 The build

Dynamo comprises a public escrow contract family, open client packages, gateway tooling, and conformance artifacts. The stream engine and several merchant and evidence components are proprietary [32-35, 44]. Public client code makes the interface inspectable without making the full runtime independently reproducible. Feature availability follows the component and integration boundaries described below.

The contract suite includes `BudgetVault`, an add-only `AssetRegistry`, and a Chainlink-shaped price adapter. The public build configuration pins Solidity 0.8.24 with optimization enabled at 200 runs. The public SDK is `@dynamoprotocol/sdk` 2.1.0, used with owner API 1.1.0 [34, 35, 38]. Integrations must check the configured instance's reported interface and funding modes, rather than infer them from a package name alone.

DreamCade and **DreamReel** illustrate integration into an existing AI application's delivery path. DreamCade generates and tests a browser game, including bounded repair work. DreamReel generates a video series with metering around episode and shot production. The commercial purpose is to show how an application can connect delivery to payments without building a separate coordination system for every stage. Both are project-reported Base Sepolia integrations using testnet assets [36].

The site's ten-line starting point refers to a minimal first metered, paid call, not the total implementation of either application or its production controls. A reproducible ease-of-integration result must state its starting scaffold, dependencies, credentials, funding setup, changed executable lines, and time to a successful reconciled test payment. The relevant outcomes are integration effort, reliable collection time, and retained proceeds after total costs. An end-to-end pass count alone establishes none of them.

7.2 The package map

Table 3. Components, access, and implementation scope.

Package or component	Stated license/access	Role and limitation
`contracts`	BUSL-1.1, public source	Vault, registry, price adapter; directly inspectable contract semantics
`engine`	Proprietary	Budgeting, metering, backends, higher-level policies; runtime behavior is project-described
`funding-card`	Proprietary	Card backend; documented implementation restricts keys to TEST mode
`@dynamoprotocol/sdk`	Apache-2.0	Published thin HTTP client, v2.1.0; does not embed the core
`plugin-litellm`	Apache-2.0	Python gateway integration, published as `litellm-dynamo`
`gateway-kong`, `gateway-apisix`	Apache-2.0	Lua integrations; a shared vendored decision core reduces implementation independence
`mcp`	Apache-2.0	MCP monetization adapter
`@dynamoprotocol/guard`	Apache-2.0	Capped proxy for compatible clients

Package or component	Stated license/access	Role and limitation
`conformance-*`	Apache-2.0	Public protocol-specific checks and test vectors
`events`, `merchant-api`	Proprietary	Webhook outbox and payee-facing verification interfaces
`evidence`, `console`	Proprietary	Evidence assembly and read models; operator-mediated export; no public-client export method

The licensing boundary is component-specific [44]. Publicly inspectable source is not necessarily permissively licensed: BUSL-1.1 has use restrictions before its change terms apply. Runtime exposure is also separate from package existence; not every engine capability is available through the published client.

7.3 Test surface

Reported test coverage comprises approximately 490 TypeScript tests across 16 packages, 170 Solidity tests, 34 Python tests, 259 Lua assertions per pinned gateway image, at least 22 property suites with 10,000 randomized runs each, and a 10,000-vector decision comparison across four plugin implementations in three languages. These are project-reported inventory and campaign figures. They describe the intended evaluation surface, not an independent certification or a proof of complete state-space coverage.

The public Foundry configuration specifies **10 invariant functions x 2,100 runs x depth 100 = 2,100,000 configured handler-call opportunities**, with 14 handler selectors in the vault harness. Configuration is not an execution log: early termination, reverts, framework behavior, and campaign structure can change meaningful transition coverage. In particular, `fail_on_revert = false` requires successful and rejected calls to be reported separately [34].

Table 4. Contract invariant families and their scope.

ID	Property family	Required qualification
I1	Network/subtree caps and accounting conservation	Gross counters; modeled transfer semantics; Proposition 1
I2	Rate-envelope conformance	Declared timestamps and configured clocks, not measured instantaneous burn
I3	Refund bound/idempotence	Envelope excess and available payee funds, not full recovery of every failed delivery
I4	Revocation cutoff	Rejection of timestamps at/after revoke, not all post-revoke submissions
I5	Subtree revocation ordering	Successful transaction and executable subtree size
I6	Replay rejection	Persistent stream high-water state and correct domain/bindings
I7	Scoped-key confinement	No cross-stream key reuse or owner-key compromise
I8	Oracle fail-closed checks	Defined invalid inputs, not universal price correctness
I9	Exact asset outflow conservation	Include both payee and collector transfers and asset rounding
I10	Batch/sequential equivalence	Equivalent initial states and successful atomic execution

ID	Property family	Required qualification
I11	Administrator restrictions	Bounded fee/collector powers remain; not literally no power

Property-based testing searches sampled execution paths; it does not prove universal correctness. The mathematical propositions in Section 5 are conditional arguments about the stated transition system, not machine-checked proofs of the compiled implementation. Engine unit and property tests using in-memory databases also do not establish file-backed WAL durability under arbitrary crashes. Crash testing requires the actual persistent-storage configuration.

Reproducibility manifest. A reproducible result identifies repository and commit, dependency lockfile, compiler and runtime, operating system, chain and deployment address where relevant, seed, test distribution, trial count, successful and rejected operations, invariant definitions, full logs, and artifact hashes. Cross-language agreement also requires an independent reference specification and boundary and adversarial vectors. Agreement among copies of the same logic is useful but is not independent algorithmic validation. Appendix C defines the evidence boundary.

7.4 The integrator contract, the sustainability harness, and the conformance methodology

Integrations that touch settlement are evaluated against nine acceptance requirements [42].

Requirement	Acceptance criterion
R1: Event idempotence	Deduplicate a stable authenticated event identifier durably, including crash/retry cases
R2: Credit only on finality	Credit only the particular verified final frontier; do not infer all newer claims are final from an older final row
R3: Duplicate/reorder tolerance	Apply cumulative advances without double counting; quarantine unknown backend outcomes
R4: Conformance	Name the protocol version, scheme, target implementation, and covered checks
R5: Payee-side verification	Verify authority, amount, domain, and funding independently of a caller’s unsupported assertion
R6: Per-mode onboarding	Supply the required chain/asset or PSP configuration and explicitly labeled sandbox credentials
R7: Monetary units	Keep physical service units, micro-USD accounting amounts, token base units, and card cents distinct
R8: Version discipline	Pin compatible interfaces and document additive changes and deprecations
R9: Sandbox/production parity	Reuse logical interfaces while separately testing real funding, transfer, recovery, and finality conditions

R7 requires dimensional correctness: a physical service measurement, a priced micro-USD accounting amount, an ERC-20 base-unit amount, and a card-cent amount are separate quantities even when a particular conversion is an identity. R9 requires both logical interface conformance and backend-specific testing; simulation cannot reproduce every production authorization, transfer, or recovery condition.

The integration sequence is: authenticate the request; reserve bounded authority where needed; execute; determine billable delivery under the agreed policy; meter idempotently; checkpoint; reconcile independently; and retain the evidence. A gateway may perform some operations asynchronously, but asynchronous reporting cannot substitute

for a correct pre-execution spending boundary. The plugin wire records and signs without checkpointing, so a host must drive settlement explicitly.

Fee harness. Each fixture's gas and PSP costs must be recomputed in consistent units, with the cost bearer identified and unsuccessful operations, refunds, and nonreturned charges included. Results should identify which fixtures satisfy (16), together with sensitivity to fee rate, gas price, payment size, refund fraction, and checkpoint count. The published methodology includes TEST-mode PSP fixtures; those fixtures evaluate payment arithmetic and integration, not production-money economics [40].

Conformance methodology. The public kit describes twelve checks covering challenge shape, supported schemes, denial ordering, and merchant protection. Passing establishes only the defined coverage. Reproducing the reported 10,000-vector comparison requires the generator, seed and distribution, target binaries, and authenticated results. The two Lua gateways share a byte-identical decision core: their agreement tests integration consistency but is not evidence from two independent algorithms. Compatibility claims must name Dynamo's `dynamo-allowance` surface and the relevant x402 version, scheme, and extensions [38, 44; 23-26].

Economic illustration. A USD 0.50 job with a 1% protocol fee and assumed USD 0.0016 gas costs USD 0.0066, or **1.32%**. At zero protocol fee, the same assumed gas cost is **0.32%**. A hypothetical card tariff of 2.9% plus USD 0.30 costs USD 0.3145, or **62.9%**, for one separate USD 0.50 charge. These are arithmetic consequences of specified assumptions, not a current universal tariff, a measured population average, or a comparison against aggregated card billing. Funding, bridging, conversion, failed calls, refunds, taxes, and amortization can change the result.

Empirical status. This paper presents conditional proofs, arithmetic illustrations, and project-reported implementation results. This paper does not report an independently executed benchmark campaign. Project-reported test counts must be linked to their execution artifacts before being treated as verified results. No production latency, conversion-rate, margin, or customer-retention estimate is inferred from the testnet applications.

Comparative evaluation. A controlled evaluation should compare the same service traces under Dynamo, a transactional gateway budget, current x402 methods, an MPP session, and aggregated card billing. Report cap violations, legitimate-work refusal rate, uncollectible delivered work, recovery success, p50/p95/p99 admission and settlement latency, total lifecycle cost, and evidence completeness. Vary concurrency, subtree width/depth, request size, price uncertainty, outage duration, key compromise, and dishonest observations. Demonstrations without those controls establish feasibility, not relative superiority.

7.5 Demonstrated behaviors

The reported demonstrations exercise selected failure and settlement paths. Each result is bounded by its configuration, input trace, and execution environment; none alone establishes a universal guarantee.

Reported demonstration	Evidence-supported interpretation	Present limitation
Recreated USD 47,000 runaway stopped at USD 63	A specified guarded trace stopped earlier than its matched baseline	Baseline is a recreation/counterfactual, not independently verified avoided loss; published SDK does not expose anomaly configuration
`kill -9` billing at the last signed voucher	Crash recovery for the tested injection point and storage setup	Does not cover every chain/PSP/local commit boundary or budget rehydration
One authorization, two payees, cent reconciliation	Capture, split, refund, reversal arithmetic for that processor session	Distinguish PSP TEST mode from production charges and name capture capabilities

Reported demonstration	Evidence-supported interpretation	Present limitation
Renewal decline freezes dependent streams	No admitted new usage after the tested unfunded renewal	Requires state-result handling and timing/concurrency coverage
Stale/deviant price rejection	Rejection of the tested stale/out-of-band observations	Does not prove all oracle manipulation is prevented or all collateral remains redeemable
71 offline evidence checks; one-byte change produces three failures	Verification of the exact bundle, trust root, and tamper fixture	Does not by itself prove provenance, completeness, legal acceptance, or current finality

Each demonstration requires an executable script, input trace and configuration, pinned dependencies, expected result, full output, and a reproducible linkage to the tested build. Testnet and production claims additionally require actual chain or PSP references and confirmation rules. For an application workflow, the evidence includes payment and refund explorer records, a verbatim signed receipt, and the aggregate reconciliation result. A product page or pass count cannot substitute for this replication package.

7.6 Deployment shapes

Deployment options are a self-hosted licensed core and dedicated hosted instances. Multi-tenant operation is a future architecture rather than an implemented deployment mode. The public site reports Control Mode and Base Sepolia escrow use; production card and mainnet activation remain per-instance and review-gated [36]. Activation controls must be bound to the actual runtime image, configuration, and environment.

Dedicated hosting isolates instances but does not automatically remove operator access to signing material. Self-hosting shifts operational responsibility to the user. Multi-tenant support would require tenant-scoped keys, authorization, durable budgets, resource isolation, and demonstrated cross-tenant failure containment; it is not a labeling change.

Performance evaluation distinguishes local credential verification, an HTTP metering call, voucher creation, checkpoint inclusion, and finality. Offline credential verification can remove a remote authorization round trip; it still incurs local processing and does not establish zero overhead for the complete request path. Simulated application measurements, isolated timings, and testnet results must not be presented as production latency distributions.

8. The Interplanetary AI Economic and Financial Layer

8.1 Architectural scope

Dynamo’s ambition is to make economic coordination a native capability of AI systems. An agent should be able to acquire inputs, deliver a service, receive payment, and account for its obligations within a framework that users and counterparties can inspect. Budgets and payment streams are enabling mechanisms. The larger opportunity is a network of independently operated services exchanging intelligence, compute, data, and other measured resources.

For providers, the entry proposition is concrete: receive payment sooner, reduce payment overhead where the workload permits it, and retain more of each transaction. For users, shared authority and delivery records make multi-service workflows easier to control. These interests reinforce one another when clear limits make buyers more willing to authorize work and predictable collection makes sellers more willing to serve them. This is a commercial hypothesis, to be evaluated through independent counterparties, repeated use, and cost and recovery outcomes.

The protocol need not become a marketplace to support one. Discovery, search ranking, identity, contract formation, and reputation can be supplied by applications or complementary standards. Settlement history may support those functions, but a transfer between colluding or commonly controlled accounts is not proof of independent demand. Dynamo's durable differentiation must come from integration quality, reliable coordination, useful evidence, and trusted operating practices rather than from a claim that budget counters or payment APIs cannot be copied.

8.2 The economic functions

Function	Dynamo foundation	Additional requirement
Service commerce	Metered claims and programmable settlement	Discovery, agreed prices, cancellation terms, and measurement provenance
Multi-provider coordination	Parent budgets and delegated streams	Correct allocation of dependencies and concurrent obligations
Bookkeeping	Usage records, vouchers, and transfer references	Reconciliation of external invoices, refunds, liabilities, and asset values
Escrow and recourse	Contract-held funds and breakpoint mechanisms	Funded holdbacks and adjudication where stronger recovery is required
Exposure management	Limits on unsettled claims	Counterparty analysis, collection rights, and available liquidity
Portable evidence	Signed records and digest manifests	Independent trust roots, completeness, and procedure-specific acceptance
Reputation and participation	Attributable service and settlement history	Identity continuity, independent activity, and resistance to fabricated use

The architecture supplies primitives for economic institutions, not replacements for every accounting, credit, assurance, or dispute function. Its contribution is their composition with bounded delegation. A common data model can reduce inconsistent records and duplicated integration work, while each funding backend retains its own operational and finality assumptions.

8.3 Integration and deployment sequence

An application integration starts by defining a billable unit, measurement source, tariff, authority boundary, and failure policy. It then authenticates and reserves authority where necessary, executes the service, records eligible usage idempotently, checkpoints a claim, and reconciles the backend outcome. Separate testing of these transitions matters more than the size of the first code sample. A minimal integration does not remove the need for provider-invoice accounting, retries, refund handling, or key management.

Control Mode exercises the metering and authority model without moving funds. Testnet escrow and processor sandbox modes exercise their respective payment paths. A production deployment requires the actual funding, transfer, finality, and recovery assumptions to hold. Common object shapes reduce interface changes, but do not establish equivalence between a simulated ceiling, contract-held tokens, and a processor authorization.

DreamCade and DreamReel provide project-reported application examples. Their technical evaluation should identify which generation stages are metered, which failures are billable, how a job's budget is allocated, and how signed claims reconcile with transfers. The ten-line starting point is a minimal-call interface claim to test with its prerequisites, not a substitute for measuring full integration effort.

A multi-provider workflow additionally requires an unambiguous mapping from service dependencies to the authority tree. Shared dependencies, joins, and cycles cannot be charged twice merely because they occur in multiple branches. Independent payer and payee control, concurrent reservation tests, and collection failures belong in an end-to-end experiment. Control-only use remains possible without external seller adoption; cross-party settlement requires agreement on the relevant service and funding terms.

8.4 Resource cost and settlement cadence

Settlement policy trades transaction overhead against delayed payment and outstanding exposure. Equation (25) gives a conditional optimum for a simple constant-rate cost model; it is not an empirically calibrated scheduling algorithm. Variable workload, gas prices, inclusion delays, and unavailable counterparties require constrained or adaptive policies. Section 7 specifies the measurements needed to compare them.

The technical evaluation separates four quantities: service value, protocol fees, costs of executing settlement, and costs of producing the service. Equation (16) tests only whether the modeled fees cover the modeled settlement costs. It does not establish that the service or protocol can fund all operating and security obligations. Repeated tiny settlements can also lose fee revenue through flooring under Equation (15), even when cumulative billed usage is accurate.

A comparison must hold the service trace and required guarantees constant. A separate card charge per unit, an aggregated card payment, and a token settlement batch have different fixed costs, collection timing, and recourse. Appendix D supplies explicit accounting definitions and arithmetic examples. Those examples motivate evaluation conditions; they are not presented as observed comparative results.

8.5 Shared infrastructure and a possible dedicated ledger

A dedicated ledger is a possible response to observed settlement load, predictable fees, governance requirements, or shared infrastructure needs. It is not required to validate the initial service proposition. Compare batching, checkpoint improvements, existing rollups, and other execution environments before assuming a new chain is superior.

The comparison must account for validator incentives, data availability, censorship resistance, bridges, asset liquidity, recovery, and integration cost. Host-chain fees purchase security and execution; redirecting them does not remove those responsibilities. Neutrality depends on inspectable rules, bounded powers, reliable operation, and credible migration rights rather than a token’s existence alone.

Section 9 gives DYNAMO a prospective native-gas role if a ledger is justified. That role must be considered together with a specified consensus and security budget. The stable accounting denomination of a service remains distinct from the asset used for gas. No dedicated-ledger launch decision or performance threshold is established by this paper.

8.6 Space based AI and the interplanetary extension

Space-based AI is already a subject of hardware demonstrations, operating missions, and infrastructure research. These are different maturity levels, and they justify a precise opportunity rather than a claim that an interplanetary market is already large or that Dynamo already serves it.

Training and inference demonstrations. Starcloud reports that Starcloud-1 launched with an NVIDIA H100 in November 2025 and subsequently trained nanoGPT and ran language-model inference in orbit [46]. This is a project-reported demonstration of orbital AI workloads, not proof of the economics of frontier-scale training or a commercial service-level agreement. ESA reports that PhiSat-2 completed commissioning and entered its science

phase in 2025, using onboard AI to process Earth-observation imagery [47]. That is evidence of useful onboard inference and data processing, with a mission-specific workload and operating context.

Data-center research. Google's Project Suncatcher studies satellite constellations carrying TPUs and optical inter-satellite links. Its published work examines communication, formation dynamics, radiation, and economic feasibility while identifying unresolved engineering requirements [45]. This supports treating orbital data centers as an active research direction. A concept, an announced mission, and a demonstrated economically competitive data center remain distinct claims.

Dynamo's proposed role is the economic coordination of services running on such infrastructure: paying for eligible compute-seconds, inference jobs, data products, storage, or relay capacity under explicit authority and terms. Dynamo does not supply launch vehicles, orbital power, hardware reliability, or proof that those services are economically viable. Space-based infrastructure is the application context; the interplanetary economic and financial layer is the ambition for coordinating value among its participants.

The following are proposed application mappings, not Dynamo deployments or partnerships.

Workload	Possible commercial unit	Measurement and operational requirement
Remote AI training	Accepted compute interval or checkpoint artifact	Defined hardware allocation, checkpoint integrity, and treatment of failed runs; training accuracy is a separate criterion
Orbital inference	Completed request or eligible compute time	Input/output commitments, measured availability and latency, and explicit acceptance conditions
Data-center services	Compute, storage, or bandwidth consumed	Capacity allocation, metering provenance, service availability, and resource pricing
Observation and relay services	Licensed data product or delivered bytes	Rights to supply the data, completeness and integrity checks, and delayed delivery rules

The physical boundary. Power availability depends on orbit and system design. Heat rejection requires engineered thermal systems; vacuum does not make cooling free. Training jobs requiring tight synchronization differ from inference close to a sensor or a batch workload with relaxed timing. Propagation delay, ground-contact schedules, radiation, launch and replacement costs, and utilization affect the business case. Large projected space-compute capacity is not automatically Dynamo's addressable payment volume.

The distributed-systems boundary. Delay-tolerant networking provides a store-and-forward model for intermittent links [31]. A disconnected verifier can check records against previously trusted keys, but cannot learn a remote revocation or a different partition's latest spend without communication. A terrestrial oracle-freshness limit can halt legitimate remote work. Low-Earth-orbit demonstrations also do not validate a consensus protocol across planetary distances.

A possible research design preallocates disjoint local authorities $C_1, \ldots, C_m$ satisfying

$$\sum_{j=1}^{m} C_j \leq C. \qquad (26)$$

Each domain enforces its allocation locally and reconciles on reconnection. Safety requires non-overlapping authority, domain-bound credentials and vouchers, and a rule preventing authority from being reused while a disconnected domain can still exercise it. Expiration and revocation require bounded-clock assumptions or a

specifically asynchronous design. This trades utilization and flexible reallocation for partition safety. It is not instantaneous global revocation or offline global consensus.

An initial space-oriented validation program can use recorded contact schedules and fault injection before any flight deployment. It should measure allocation utilization, legitimate-work refusals, maximum outstanding exposure, duplicate-claim rejection, reconciliation completion, and recovery after key or clock faults. Later mission tests would require a named partner, hardware and software configuration, independently checkable results, and an economic comparison. No such Dynamo flight result is asserted here.

The development sequence is **demonstrate useful services, validate disconnected operation, then extend across space**. Terrestrial commercial utility supports the project before orbital demand develops. The same economic vocabulary can extend outward only as the required authority, measurement, communication, and settlement mechanisms are established.

8.7 How the utility design relates to the vision

The DYNAMO utility thesis attaches the token to defined participation and infrastructure functions. Section 9 specifies prospective governance, ecosystem participation and trust bonds, and gas for a possible dedicated ledger. These roles address different coordination needs: bounded policy decisions, accountable participation, and payment for execution. The trust-bond proposal grants additional risk allowance rather than basic access and earns no yield.

The escrow rail can operate with supported settlement assets without requiring buyers or sellers to acquire DYNAMO. This allows application adoption to be evaluated on its own merits. The distinction between a service settlement asset and an ecosystem utility asset preserves familiar pricing for users while leaving the stated token roles tied to actual mechanisms and their readiness requirements.

A credible utility anchor is operational: identify who uses each function, the action they perform, the rule the system enforces, and evidence that the function is useful. Governance proposals require executable scope; bonds require a specified risk and recourse mechanism; a gas role requires a live, independently justified ledger. Section 9 specifies the functions and their activation requirements. The token is an instrument for specified protocol functions; the interplanetary scope concerns where those functions may be needed and which operating assumptions they must satisfy.

9. The DYNAMO Utility Token

DYNAMO is the proposed utility token of the Interplanetary AI Economic and Financial Layer. Its role is to support the operation of a network of independent AI services: participation in bounded protocol decisions, accountable service provision, and payment for ledger execution if dedicated infrastructure is justified. The same functions can support terrestrial and space-based services when their operating requirements are met.

The protocol and the token are separate components. Dynamo’s payment protocol can operate with supported settlement assets without requiring a DYNAMO balance. A micro-USD accounting amount measures an obligation; a settlement asset transfers payment; DYNAMO has the protocol functions specified below. Token possession alone does not authorize a service call, establish a budget, authenticate a measurement, or verify delivered work.

These token functions are proposed. The public vault’s accounting and authorization rules do not by themselves implement token governance, a trust-bond system, or a dedicated Dynamo ledger. Each function requires its own executable specification and evidence before it can be described as available.

9.1 Functional scope

The design has three utility roles. **Governance** concerns bounded protocol policy. **Participation credentials and risk bonds** associate accountable operators with defined responsibilities and exposure limits. **Gas** pays for transaction execution if Dynamo operates a dedicated ledger. Basic service access continues to depend on the service's terms, funded authority, and valid credentials.

A token role must identify the participant using it, the operation performed, the state the system records, and the rule that operation enforces. A balance is not a substitute for an identity binding or an authorization. Work credentials must distinguish the identity of a contributor, evidence of completed work, and any token allocation associated with participation. Transferring tokens must not transfer another operator's work history or signing authority implicitly.

9.2 Bounded protocol governance

Governance allows eligible participants to take part in decisions about shared protocol policy. Candidate subjects include bounded fees, admission policies for new settlement assets, oracle standards, holdback and exposure defaults, and deployment or migration policy. Such decisions matter when independently operated services need compatible accounting and evidence rules.

The intended protected properties are restrictions on administrative access to principal, cap conservation, fail-closed settlement checks, and evidence integrity. Governance must operate within technically enforced boundaries. A vote cannot change an immutable contract by declaration or safely bypass obligations already committed under its rules. An unrestricted upgrade authority would undermine those boundaries.

The public vault has owner-controlled fee settings, not a DYNAMO voting mechanism. Its asset registry is add-only: a policy decision cannot edit an existing listing through a nonexistent setter [32, 33]. Changing those semantics requires an explicit deployment or migration path. The governance specification must define proposal eligibility, participation measurement, voting and execution rules, conflicts of interest, activation timing, and the treatment of existing sessions. The design principle that participation matters requires an enforceable mechanism; transferable-token voting alone does not establish it.

In a space-based service domain, governance could coordinate supported measurement standards, accepted attestations, or exposure policies. A disconnected operator must authenticate and receive a policy before enforcing it. Policy activation therefore requires a domain identifier and explicit timing and reconciliation rules. A remote vote does not create instantaneous global policy propagation.

9.3 Participation credentials and risk bonds

Participation records identify contributions by service providers, integrators, and other protocol operators. A work credential binds an attributable participant to evidence under defined validation rules. It must identify its issuer, covered activity, expiry or revocation behavior where applicable, and verification method. Cryptographic integrity establishes who signed the record, not whether the underlying contribution was useful or honestly reported.

A proposed DYNAMO trust bond is collateral attached to a defined operating responsibility. Its functional purpose is to support an additional **risk allowance, not basic access**, and it **earns no yield**. An oracle operator, or a validator in a specified future ledger, could post collateral against defined misconduct. The relevant roles require separate specifications: a service risk bond and a consensus-security bond do not cover the same failures.

For a service risk bond, the required state includes the bonded principal, responsible operator, permitted exposure, beneficiaries, covered events, pending claims, and release conditions. The mechanism must specify the source and admissibility of evidence, loss allocation, challenge procedure, withdrawal delay, and execution of an upheld

claim. User-owned evidence and a public record of attributable operating history are proposed prerequisites. These requirements describe a mechanism to implement, rather than a claim that the public vault supplies it.

Additional exposure must be justified by conservatively valued, enforceable collateral. A native-token balance can lose effective collateral value at the same time the operator's obligations become harder to satisfy. Valuation haircuts, concentration limits, and a response to inadequate coverage are therefore necessary operating parameters. A token amount alone does not establish a safe monetary allowance. The comparison must also consider settlement-asset collateral or a lower exposure limit; the use of DYNAMO must have a demonstrated protocol purpose.

For remote inference or observation-data services, a bond could support accountable attestation or bounded unsettled work. Expected link outages must be distinguished from misconduct. Release timing must accommodate the communication and challenge windows; an unreachable observer cannot supply immediate evidence. The same collateral must not be committed to overlapping obligations in disconnected domains. A bond does not independently verify model quality, compensate every service failure, or guarantee mission reliability.

9.4 Gas for a dedicated ledger

If measured settlement requirements justify a dedicated Dynamo ledger, DYNAMO could pay for transaction execution on that ledger. The functional unit is execution and inclusion of a transaction, not a share of a service provider's revenue. An EIP-1559-style base-fee mechanism is one possible design [7]; the consensus protocol, fee accounting, validator compensation, congestion behavior, and security budget require specification together.

Service tariffs remain separate from gas. A provider may price a workload in compute-seconds, tokens processed, frames, or delivered bytes, then settle the resulting obligation using an accepted asset. Gas pays for the ledger operation recording or executing that settlement. An application must identify who supplies gas, how its cost is bounded, and what happens when the balance is insufficient. A service budget excludes separately paid gas unless the integration explicitly accounts for it.

The architectural decision must compare existing chains, rollups, batching, and checkpoint policies against a dedicated ledger. Required evidence includes actual transaction demand, inclusion latency, execution costs, fault recovery, and an adequate security budget. A token does not itself justify another chain. This role remains inactive until the corresponding ledger exists and its execution interface is defined.

9.5 Operation across space

Orbital training and inference demonstrations and data-center research establish a concrete application context [45-47]. They do not establish a Dynamo deployment. The protocol's prospective users include operators exchanging training checkpoints, inference results, observation data, storage, or relay capacity. The token's role is to support the protocol functions used in those exchanges, rather than to represent the physical resources themselves.

The domain-allocation rule in Equation (26) gives a starting point for disconnected operation: allocate non-overlapping authority, enforce it locally, and reconcile after communication resumes. Token-related state needs the same discipline. A local authorization must not imply that a global token balance has changed; a bond must not be released while another domain can still exercise a covered claim; and policy revisions must not retroactively invalidate accepted work without an agreed rule.

An initial remote workflow can retain ledger execution on connected infrastructure while recording signed work evidence in the remote domain. Local token transfer or remote consensus would require an additional protocol specifying double-spend prevention, settlement authority, and finality during partitions. Neither offline signature

verification nor token ownership supplies that protocol. Introducing remote execution must be justified by requirements that ordinary delayed submission cannot meet.

The development path is to demonstrate each function in terrestrial operation, test delayed and interrupted communication with controlled fault injection, and then evaluate a specified mission integration. Contact schedules, key compromise, clock faults, policy propagation, evidence delivery, and collateral-release races belong in those tests. The utility remains useful for terrestrial AI services without depending on a future orbital market.

9.6 Activation and conformance

Each deployed role must publish its state transitions, authorization boundaries, failure behavior, and reproducible conformance tests. Governance tests must include unauthorized execution and conflicting activation states. Credential tests must cover incorrect identity bindings, fabricated work records, replay, and revocation. Bond tests must cover concurrent commitments, inadequate collateral, conflicting claims, and premature release. A ledger implementation must test fee accounting, insufficient gas, transaction reordering, and chain recovery.

Shared identifiers must bind these functions to the correct instance, domain, participant, and policy. Operational records must distinguish available authority from committed obligations and identify which system is authoritative during recovery. The protocol's accounting proofs do not automatically extend to a token implementation: its additional transitions and trust assumptions must be analyzed explicitly.

The technical criterion for DYNAMO utility is whether a participant can perform a specified protocol operation with an observable, enforceable result. Governance, accountable participation, and execution can serve that purpose as their implementations mature. Their applicability across space depends on the same authority, measurement, and recovery requirements that govern the underlying AI economic and financial layer.

10. Discussion

Audit and evidence status. Public contract code and test configuration make the settlement rules inspectable, while proprietary runtime components remain outside the public source boundary. Production money-rail activation remains subject to independent security, legal, and PSP/PCI review [36]. The formal arguments and reported tests in this paper are not an independent security audit. A passing fuzz campaign provides empirical evidence under its harness rather than a universal proof.

Operator centralization. Dedicated hosting and self-hosting place operational responsibility in different hands. Hosted operation relies on the operator's configuration, key handling, software supply chain, durable state, and availability. The project-reported reference runtime identifies limitations in restart-related budget reachability, failed-revocation retries, and finality scheduling; each must be resolved or explicitly contained in a production deployment. They separate settlement safety from liveness: labeling a payout pending does not keep transferred tokens in the vault, and a zero holdback setting does not remove blockchain or card-network finality risk.

Objective facts are a limited foundation. Authenticated unit counts, timestamps, acceptance messages, and service observations can support auditable payment decisions. They do not make the observer infallible or turn subjective quality into an objectively established fact. Richer validation markets may complement the protocol, provided their claims, incentives, and failure modes are explicit. ERC-8004's validation interface is relevant prior work, but its existence is not proof that any particular validator or market is trustworthy [12, 13].

Deliberate and accidental restrictions. One-way cap allocation and same-second timestamp clipping are explicit current behaviors. They can simplify accounting or favor the payer, while reducing utilization or making completed work uncollectible. Their merits should be evaluated rather than assumed. Client-local metadata is not

durable correlation infrastructure. A future implementation can change these choices, but must identify which invariants and compatibility promises change with them.

The agent-as-buyer premise. The strategic thesis assumes growing use of software acting under delegated spending authority. The architecture remains applicable to ordinary automated jobs and APIs even if fully autonomous commerce develops more slowly. That supports developer monetization and ordinary automated-service workflows as well as spending control; it does not establish that the broader economic layer will be necessary, dominant, or commercially successful. Adoption evidence should separate independent users, repeat workflows, real payments, testnet activity, and subsidized or self-generated volume.

Open problems. Four directions extend the architecture. First, *validation-market integration* requires explicit measurement provenance and accountable validation. Second, *multi-chain expansion*, including non-EVM implementations, must preserve accounting and domain separation while specifying bridge and finality assumptions. Third, *interoperability* requires conformance with named x402, MPP, AP2, and network-program specifications; shared vocabulary or an HTTP 402 response does not establish compatibility. Fourth, *decentralized oracle operation* needs a defined bonded set, objective misconduct rules, collusion resistance, and availability analysis.

Near-term validation priorities are a public claim-to-artifact manifest; cap and reservation concurrency; crash injection at every external-effect/local-commit boundary; revocation and closure races; backdated vouchers; refund-funding failures; cold-start recovery; and end-to-end finality reconciliation. Evaluation should report hardware, software identifiers, workload distributions, repetitions, dispersion, failed cases, and the distinction between local processing, metering round trip, inclusion, and finality. Broader deployment and economic claims depend on this evidence.

11. Conclusion

Dynamo is the Interplanetary AI Economic and Financial Layer: a vision for AI services and agents to earn, spend, and coordinate value across borders and across space. Its immediate proposition is useful to developers and users without depending on the full vision being realized. Developers can connect metered delivery to payment, evaluate lower transaction overhead, and coordinate service inputs. Users can allocate one budget across agents and inspect the resulting usage and financial records.

The protocol’s current foundation is precise. Under the stated contract model, gross accounting settlement respects stream, subtree, and aggregate caps, while cumulative vouchers prevent duplicate settlement of the same increment. End-to-end spending control, reliable payee collection, truthful quality observations, and refund recovery require additional integration and operating assumptions. Those distinctions make the claims testable rather than weakening their practical value.

The business case must be demonstrated through integration effort, independent repeated use, collection time, and retained proceeds after total costs. Orbital AI training, onboard inference, and data-center research provide a grounded context for extending the architecture, while long-delay operation introduces separate technical and economic requirements. The DYNAMO utility design connects prospective participation and infrastructure functions to this development path; it does not replace the need for useful services and sustainable operation.

One budget. Many agents. Payment streamed by the second. Across the universe. The ambition is broad; its credibility rests on explicit mechanisms, reproducible evidence, and benefits that counterparties can measure.

References

References identify the protocol implementations, research, and technical standards supporting this paper. The documentary scope is September 2026. Source-code claims refer to the immutable snapshot specified below, rather than an unspecified live deployment. Mutable documentation should be archived with any replication package.

[1] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. Source.

[2] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, 2014. Source.

[3] J. Poon and T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. 2016. Source.

[4] R. Bloemen, L. Logvinov, and J. Evans. *EIP-712: Typed Structured Data Hashing and Signing*. Ethereum Improvement Proposals, 2017, specification. Source.

[5] F. Vogelsteller and V. Buterin. *ERC-20: Token Standard*. Ethereum Improvement Proposals, 2015. Source.

[6] S. Ellis, A. Juels, and S. Nazarov. *ChainLink: A Decentralized Oracle Network*. 2017. Historical background reference. Source.

[7] T. Roughgarden. *Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559*. arXiv:2012.00854, 2020. Source.

[8] P. Daian et al. *Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges*. IEEE Symposium on Security and Privacy, 2020; arXiv:1904.05234. Source.

[9] E. Reppel et al. *x402: An Open Standard for Internet-Native Payments*. x402 whitepaper, May 2025. Historical version; current capabilities are cited separately in [23-26]. Source.

[10] D. Rothschild et al. *The Agentic Economy*. Microsoft Research publication record, May 2025. Source.

[11] A. Vaziry, S. Rodriguez Garzon, and A. Kupper. *Towards Multi-Agent Economies: Enhancing the A2A Protocol with Ledger-Anchored Identities and x402 Micropayments for AI Agents*. arXiv:2507.19550, 2025. Source.

[12] X. Xiong, Z. Li, W. Wei, Q. Wang, W. Knottenbelt, and Z. Wang. *Can Trustless Agents Be Trusted? An Empirical Study of the ERC-8004 Decentralized AI Agent Ecosystem*. arXiv:2606.26028, version 2, 8 July 2026. Observations concern the study's sampled period, not all later deployments. Source.

[13] M. De Rossi, D. Crapis, et al. *ERC-8004: Trustless Agents*. Ethereum Standards-Track proposal; draft specification. Source.

[14] T. Kwa, B. West, J. Becker, et al. *Measuring AI Ability to Complete Long Software Tasks*. arXiv:2503.14499, 2025; METR's accompanying research explanation, 19 March 2025. Source; Source.

[15] Gartner. *Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027*. Press release, 25 June 2025. Forecast, not an observed rate. Source.

[16] OWASP. *AI Agent Security Cheat Sheet*. Official threat taxonomy and mitigation guidance, including the Agentic Top 10 mapping. Source.

[17] Anthropic. *Introducing the Model Context Protocol*. 25 November 2024. Source.

[18] Stripe and Tempo. *Machine Payments Protocol*. Launch announcement, 18 March 2026, and current protocol site. Source; Source.

[19] Google and AP2 contributors. *Agent Payments Protocol*. Source.

[20] Sablier. *Protocol Documentation*. Current token-streaming documentation. Source.

[21] M. Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657, version 3, 2025. Source.

[22] X. Qin et al. *BlindHub: Bitcoin-Compatible Privacy-Preserving Payment Channel Hubs Supporting Variable Amounts*. IEEE Symposium on Security and Privacy, 2023; IACR ePrint 2022/1735. Source.

[23] x402. *Introduction*. Current official documentation. Source.

[24] x402. *Upto Scheme*. Maximum-amount authorization and usage-based settlement. Source.

[25] x402. *Batch Settlement*. Current official scheme documentation. Source.

[26] x402. *Offer and Receipt*. Signed offer/receipt extension. Source.

[27] AP2 contributors. *Agent Payments Protocol Specification*, v0.2. Source.

[28] Visa. *Trusted Agent Protocol: Documentation*. Current official developer documentation. Source.

[29] Mastercard. *Verifiable Intent*. Official explanation of the initiative developed with Google, 5 March 2026. Source.

[30] Stripe. *Multicapture*. Current processor documentation; account capability and payment-method limitations apply. Source.

[31] IETF. *Bundle Protocol Version 7*. RFC 9171, January 2022. Source.

Dynamo implementation and documentation

The implementation references below identify public source at commit `344eb61730a90f097ad3fadf7d6b20f0eec9a520`, dated 7 September 2026, in the `DynamoProtocol/dynamo-open` repository.

[32] Dynamo Protocol. *BudgetVault*. Settlement, delegation, revocation, refunds, closure, and bounded fees. Source.

[33] Dynamo Protocol. *AssetRegistry*. Asset configuration and accounting conversions. Source.

[34] Dynamo Protocol. *Foundry Configuration*. Compiler, optimization, and invariant-campaign settings. Source.

[35] Dynamo Protocol. *SDK Package Manifest*. `@dynamoprotocol/sdk` 2.1.0. Source.

[36] Dynamo Protocol. *Dynamo*. Public project and application information. Source.

[37] Dynamo Protocol. *Documentation*. Source.

[38] Dynamo Protocol. *SDK Reference and Overview*. Source; Source.

[39] Dynamo Protocol. *Evidence*. Evidence-bundle structure, access, and verification. Source.

[40] Dynamo Protocol. *Fees*. Fee schedule and settlement-cost methodology. Source.

[41] Dynamo Protocol. *Holdback and Refunds*. Design documentation; implementation boundaries are specified in Section 5.8. Source.

[42] Dynamo Protocol. *Integrator Acceptance*. Source.

[43] Dynamo Protocol. *Public Demo*. Source.

[44] Dynamo Protocol. *Public Repository Overview*. Component and licensing boundaries. Source.

Space infrastructure references

[45] T. Beals and Google Research. *Exploring a space-based, scalable AI infrastructure system design*. 4 November 2025. Project Suncatcher research overview and linked technical preprint; forward-looking infrastructure design. Source.

[46] Starcloud. *Starcloud-1*. Operator account of the November 2025 launch and subsequent orbital AI training and inference demonstrations. Source.

[47] European Space Agency. *PhiSat-2 begins science phase for AI Earth images*. 15 July 2025. Commissioning and onboard AI processing report. Source.

Appendix A - Notation

Symbol	Meaning and unit
C	Budget cap in integer accounting units; one unit is one micro-USD under the engine convention
s, c_s, r_s	Stream, its subtree cap, and its own cumulative rate envelope in accounting units/second
$t_0(s), \rho_s, t_b$	Stream start, recorded revocation, and attested breakpoint times, in whole seconds
$\mathcal{T}(s), S_s, Q_s, T_B$	Subtree; own gross settled amount; subtree gross settled total; budget gross settled total
$v_k = (\text{streamId}, U_k, V_k, \tau_k)$	Voucher: cumulative meter, cumulative accounting amount, and declared timestamp
S_k, δ_k	Stream high-water state after settlement k; gross accounting increment
E_B, D_B, R_B, P_B, L_B	Current escrow, cumulative deposits, refund inflows, settlement outflows, closure releases; asset base units
A_s, H_s	Stream's own accrued and held-reservation accounting units
K, f, F_f, N_f	Basis-point denominator 10,000; fee rate; accounting fee; net payee accounting entitlement
D, n, G	Aggregate gross amount, settlement count, and explicitly denominated settlement cost floor
d, p, h_o, k	Asset decimals, price in micro-USD per whole token, haircut basis points, and $k = K - h_o$

Symbol	Meaning and unit
$T_p(a), A_p(x)$	Asset-to-accounting and accounting-to-asset integer conversion functions at fixed price/configuration
Φ_s	Accounting refund target derived from the breakpoint-clipped rate envelope
$E_{\max}, h_b, c, g, r, \lambda$	Exposure limit; holdback seconds; checkpoint interval; per-checkpoint cost; accrual rate; risk-price coefficient
q	Remaining headroom percentage used by the degradation policy

Oracle haircut h_o and holdback duration h_b are distinct quantities.

Appendix B - Demonstrated Behaviors

The following evidence requirements define the reproducibility boundary for the reported demonstrations. Claims are limited to the configuration and behavior supported by each artifact.

Reported demonstration	Evidence to preserve	Boundary of the inference
USD 47,000 runaway recreation stopped at USD 63	Baseline and guarded traces, pricing inputs, cap/rate settings, enabled hooks, commit, logs	A scenario result; not typical savings, verified prevented loss, or proof of SDK-accessible anomaly detection
`kill -9` crash billing	Crash point, storage mode, durable voucher, recovery output, backend state	One tested recovery transition; not atomicity across every external settlement and database write
One authorization and two payees reconcile to cents	Processor mode/capabilities, capture and split records, refunds, reversals, rounding residues	Arithmetic consistency for the observed session; not production underwriting or chargeback finality
Declined renewal freezes dependents	Decline result, dependency graph, timestamps, attempted subsequent usage	Refusal after that unfunded renewal under tested concurrency conditions
Stale/deviant oracle observations refuse conversion	Feed inputs, observation baseline, expected failures, separate closure/release trace	Rejection of those inputs; not general oracle honesty or guaranteed token redeemability
71 offline checks and three failures after a byte change	Exact bundle, verifier version, external trust anchor, disabled-network setup, mutation location	Authenticity/integrity checks for the bundle; not completeness, current finality, or legal acceptance

A complete replication package should distinguish unit, property, integration, sandbox, testnet, and production evidence. These categories are not interchangeable. Unexpected outcomes and known failing cases belong in the package alongside passing examples.

Appendix C - Implementation Scope and Reproducibility

Implementation reference. The public contract analysis refers to `DynamoProtocol/dynamo-open` at commit `344eb61730a90f097ad3fadf7d6b20f0eec9a520`, dated 7 September 2026. The core source references are `BudgetVault.sol`, `AssetRegistry.sol`, the Foundry configuration, and the SDK manifest [32-35]. SDK and backend deployments must be matched to their actual interface and contract identifiers.

Evidence categories. Contract properties refer to the specified executable transitions and their assumptions. Mathematical propositions establish conditional results about that model. Configured test profiles identify a campaign design; execution logs establish observed outcomes; independent replication requires a separately reproduced artifact. Product demonstrations and deployment descriptions are project-reported. Core runtime descriptions apply to the implementation scope in Section 7.2 and do not imply that proprietary components are publicly reproducible.

Replication requirements. A complete replication package identifies source and binary hashes, dependency locks, compiler and runtime, operating system, chain and addresses, configuration, seeds, input distributions, operation counts, expected outputs, actual logs, and known failure cases. Financial records distinguish accounting amounts from asset transfers and identify simulation, processor TEST mode, testnet, or production. Evidence fixtures include an externally anchored trust root and explicit canonicalization and completeness boundaries.

Safety and liveness boundaries. The accounting-cap invariant and cumulative-voucher algebra concern accepted settlement. End-to-end expenditure control additionally requires a non-bypassable admission boundary and synchronized reservations. Payee compensation requires successful submission before closure. Refund recovery requires transferable payee funds and approval unless a separately funded reserve is implemented. Hosted operation additionally requires reliable recovery, key management, and finality scheduling. These are distinct obligations and should be evaluated separately.

Research assumptions. Dedicated-ledger and interplanetary extensions are conditional research directions whose validation requires more than terrestrial contract safety. Documented orbital AI activity motivates the application context; it does not establish a Dynamo deployment or partnership. Each proposed token function requires a specified mechanism and evidence of useful operation.

Public resources. The project website, documentation, source repository, and demonstration surface provide the public entry points [36-39, 43, 44]. A component's availability, an integration's conformance, and an independent security assessment are separate forms of evidence. Implementation and deployment claims should always be interpreted within their stated artifact and operational boundaries.

Appendix D - Economic analysis and evaluation criteria

This appendix supplies the economic context for the technical evaluation. The numerical examples are illustrative calculations, not empirical results.

D.1 Provider economics and protocol revenue

The seller business case has three separable terms: lower lifecycle payment cost, shorter collection delay, and reduced integration or reconciliation effort. None is identical to an increase in underlying customer demand. Let V be gross settled service value over a stated period, ϕ the effective protocol-fee fraction, C_{rail} other payment costs borne by the provider, C_{input} service-input costs, and L_{service} refunds and losses allocated to that provider. A contribution measure is

$$\Pi_{\mathrm{provider}} = V(1-\phi) - C_{\mathrm{rail}} - C_{\mathrm{input}} - L_{\mathrm{service}}.$$

Revenue recognition, cash receipt, and this contribution measure are distinct. Lower fees improve contribution only if other costs do not offset the saving. A 1% fee leaves approximately 99% before those deductions; it is not a 99% profit margin. Where gas is paid by the provider, fee income received by a separate protocol collector does not reimburse it automatically.

Small payments and aggregation. For a hypothetical card price $\alpha x + b$ per separate charge, a token path $\phi x + g$, and additional lifecycle costs c_C and c_D, the token path is cheaper exactly when

$$(\alpha - \phi)x + b - g + c_C - c_D > 0.$$

All terms must use the same currency and period. For n service units aggregated into one card charge, the fixed cost per unit becomes b/n. A comparison that ignores this alternative exaggerates the advantage of micropayment settlement. Conversely, aggregation can delay collection or require prepayment, minimum purchases, and additional accounting; those are real trade-offs to measure.

Section 7.4's illustrative USD 0.50 job costs USD 0.0066 under a 1% fee plus assumed USD 0.0016 gas. A separate card charge at a hypothetical 2.9% plus USD 0.30 costs USD 0.3145. Aggregating 100 such jobs into one USD 50 charge reduces the card cost per job to USD 0.0175, or 3.5%. Under the same assumptions the token example remains cheaper, but the relevant gap is much smaller. Funding, conversion, cross-border charges, failed transactions, and refund treatment can change the ranking. These are arithmetic examples, not market quotations or benchmark results.

Collection time and working capital. Under steady eligible revenue flow v in currency per day and a reduction Δd in average collection delay, the amount of receivables financed falls by approximately $v\Delta d$, assuming collectability and the timing profile otherwise remain unchanged. A flow of USD 1,000 per day collected six days sooner releases approximately USD 6,000 of working capital; it does not create USD 6,000 of additional revenue. At annual financing rate i, its approximate annual carrying-cost effect is $iv\Delta d$. Provider evaluation should report delivery-to-usable-funds distributions and uncollected work, not only fast successful transactions.

Protocol economics. The documented escrow fee is one revenue mechanism; the card backend describes a separate application-fee setting, and dedicated hosting or licensed-core operation may support additional commercial contracts [40, 44]. Their prices, uptake, and realized revenue are not established here. Potential hosting, support, or integration income must not be booked as achieved revenue. Service volume, fee income, and third-party pass-through amounts are separate accounting categories.

A commercial evaluation must identify the portion of collected fees retained to operate the protocol, the party paying settlement costs, fixed and variable operating expense, security reviews, support, losses, and reserves. Sustainable operation requires a transparent funding source for these obligations.

Measures of business validation. Pilots should report time to first reconciled payment, engineering effort, repeat active sellers and buyers, independent paid workflows, collection delay, payment success, uncollected delivered value, and contribution after total costs. Compare cohorts with equivalent workloads and funding options. Separate unsubsidized activity from incentives, internal tests, and self-transfers. Publish denominators and distributions; averages alone can conceal failure rates and uneconomic small customers. These measures connect the protocol's engineering properties to a credible business case.

D.2 Claims and evidence

The following definitions translate the commercial proposition into falsifiable evaluation criteria. A deployed offer should identify its configuration and measurement window.

Claim	Evidence and interpretation	Failure condition or boundary
Faster payment	Compare delivery-to-usable-funds distributions and uncollected work against the customer's existing path	Faster signing alone, or excluding failed collections, is insufficient

Claim	Evidence and interpretation	Failure condition or boundary
Lower fees and more retained proceeds	Compare total lifecycle costs for equivalent workloads, including aggregated card billing	Added funding, conversion, gas, or support costs can remove the saving
Per-second payment streaming	Show metered accrual and checkpointed settlement at the configured cadence	Metering granularity does not establish one-second payout finality
Easy integration	Publish the minimal sample, prerequisites, changed lines, elapsed integration time, and reconciled outcome	Ten lines to a first call is not a production integration-effort guarantee
One budget across services	Demonstrate coordinated reservation and reconciliation across independent providers	Bypassed credentials and overlapping offline authority escape the model
Quality-linked settlement	Specify measured conditions, observation sources, billable units, and recourse	Signature validity alone does not establish delivery truth or fund refunds
Payload separation	Trace data flows, adapters, logs, telemetry, and third-party service calls	Counts and identifiers can be sensitive even when payloads are excluded
Cross-border reach	Name supported countries, assets, funding paths, and observed successful collections	A compatible wallet alone does not establish universal availability
Interplanetary applicability	Test domain allocation, delayed communications, measurement integrity, and recovery	Industry demonstrations are context, not evidence of Dynamo flight operation